

1-Levins.html

by Yuki Wu

Submission date: 08-Feb-2024 09:18PM (UTC+0000)

Submission ID: 224042736

File name: 1-Levins.html (852.16K)

Word count: 3956

Character count: 19234

1: Levins metapopulation model

Essentials of Ecology coding coursework

Yuki Wu

January 2024

- Introduction
- Model specification
 - Main assumptions of the model
 - Extirpation
 - Colonisation
 - Overall change in patch occupancy
- Model building
 - Calculating rate of change
 - **Question 1.1**
 - Step-by-step simulation
 - **Question 1.2**
 - **Question 1.3**
- Running the model
 - **Question 1.4**
 - **Question 1.5**
- Challenge questions - Stochasticity
 - **Challenge Question 1.1**
 - **Challenge Question 1.2**

Introduction

The aim of this worksheet is to create the *Levins metapopulation model*. Let's set the scene. Imagine a very large number of habitat patches that can each be occupied (or not) by a population of the species we're interested in. Well, actually there are an infinite number of patches strictly speaking, but don't worry that because an infinite number may be harder to imagine than just a very large number. The patches are equally well connected to each other, allowing populations to disperse between them and colonise empty patches. The Levins metapopulation model captures this system using two key parameters: colonisation rate and extirpation rate.

Model specification

Main assumptions of the model

1. There are an **infinite number of habitat patches**, which are all equal in size and equally suitable for our species to occupy.
2. Each habitat patch only has **two possible states**: occupied (our species is there) or vacant (our species is not there).
3. Patches may **produce colonists** that disperse to occupy any other patch, or they may be **extirpated** (become locally extinct).
4. All habitat patches are **equally well connected to each other** so colonists from any patch are equally likely to reach any other patch.
5. Colonisation and extirpation of any patch happen **independently**.

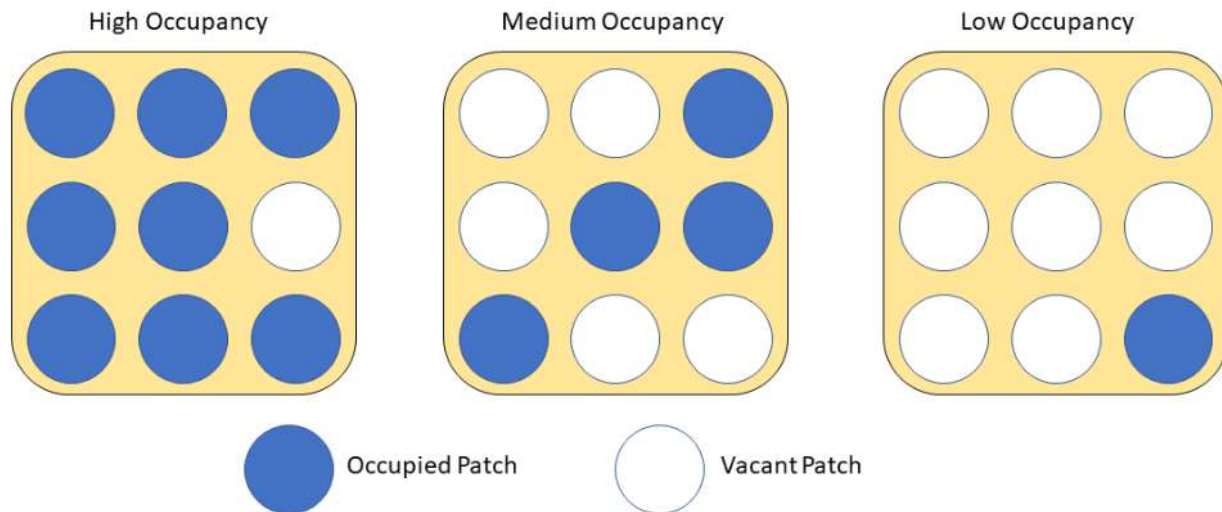


Figure 1.1: A patch occupancy model showing scenarios with different levels of patch occupancy. The Levins model looks like this, but with an infinite number of patches.

There are two main parameters in the model:

- Colonisation ($\lambda(c)$) - The rate at which a habitat patch produces colonists.
- Extirpation ($\lambda(e)$) - The rate at which the population of a patch goes locally extinct.

From these parameters we seek to predict how patch occupancy changes over time ($\lambda(p)$). This represents the **proportion** of patches that are occupied (not the number of patches!). So, if $\lambda(p=0.25)$ then one quarter of patches are occupied.

Extirpation

The local population of our species in any patch may be extirpated independently of what's going on in the other patches. Of course, a patch can only be extirpated if the species is there in the first place. So, the more occupied patches we have, the greater the rate of extirpation. The extirpation rate of patches overall is therefore proportional to the number of patches occupied:

Total extirpation rate = $\lambda(ep)$

Let's put this together with some code.

```
e <- 0.1 # Set the extirpation rate parameter (per occupied patch)
p <- 0.4 # Set the patch occupancy that we want to calculate total extirpation rate for
Ext <- p*e # Calculating the total rate of extirpation
print(Ext)
```

```
## [1] 0.04
```

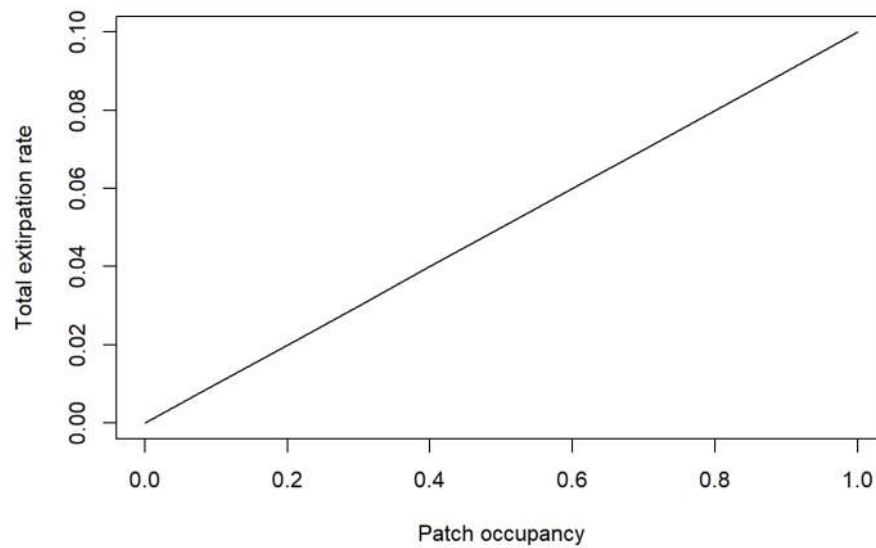
You can try editing the values of $\lambda(e)$ or $\lambda(p)$ and practice re-running that chunk to see how the total rate of extirpation changes. Now we're ready to visualise this as a graph in R.

```
e <- 0.1 # Set the extirpation rate parameter (per occupied patch)
p <- seq(0,1,by=0.1) # Creating a sequence of numbers from 0 to 1 with an interval of 0.1 between numbers to give a range of patch occupancy values
Ext <- p*e # Calculate the total rate of extirpation for each of the patch occupancies.

# Notice how, even though p is a sequence of many values we can calculate the total extirpation rate for each one with a single multiplication.

# Finally we plot the graph:
plot(p,Ext,type="l",xlab="Patch occupancy",ylab="Total extirpation rate",
     main="Figure 1.2: Extirpation rate at different patch occupancies")
```

Figure 1.2: Extirpation rate at different patch occupancies



Colonisation

The colonisation rate is proportional to both the proportion of occupied patches (p) and proportion of vacant patches ($1-p$). This is because the more occupied patches we have, the more sources from which colonisation can occur; also, the more vacant patches we have, the more destinations there are waiting to potentially be colonised.

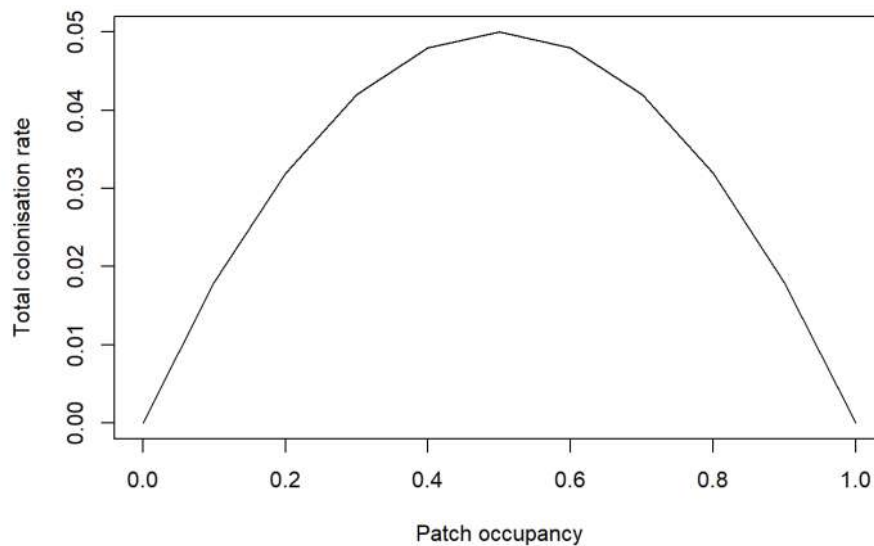
Total colonisation rate = $c \cdot p \cdot (1-p)$

Let's visualize that in R as well.

```
c <- 0.2 # Set our colonisation rate (per cell rate of producing colonists)
p <- seq(0,1,by=0.1) # Set a range of occupancy values to look at.
Col <- c*p*(1-p) # Compute the total colonisation rate.

plot(p,Col,type="l",xlab="Patch occupancy",ylab="Total colonisation rate",
     main="Figure 1.3: Colonisation rate at different patch occupancies")
```

Figure 1.3: Colonisation rate at different patch occupancies



As you can see below, colonisation rate is the lowest when patch occupancy is either very low or very high. This is because when patch occupancy is low, there are limited colonisers produced. When patch occupancy is high, there are no more patches left to colonise.

Overall change in patch occupancy

If we combine the two equations by subtracting the total extirpation rate from the total colonisation rate, we get this equation for the overall change in patch occupancy through time:

$$\frac{dp}{dt} = cp(1-p) - ep$$

The left hand side of the equation may need a bit of explaining if you've not seen a differential equation before. It gives us the rate at which patch occupancy will change over time. To get a deeper understanding we can think separately what the $\frac{dp}{dt}$ and $\frac{dp}{dt}$ parts mean. $\frac{dp}{dt}$ represents an infinitely small bit of time and $\frac{dp}{dt}$ gives the amount that p would change by over this small bit of time.

Model building

Calculating rate of change

Question 1.1

Let's first produce a function that takes three inputs (patch occupancy, colonisation, and extirpation) and outputs the rate of change ($\frac{dp}{dt}$) following this equation:

$$\frac{dp}{dt} = cp(1-p) - ep$$

Here are some requirements:

- The function must be named `RateOfChange`
- Inputs should be named `c`, `e` and `p`

```
### QUESTION 1.1 ###
```

```
RateOfChange <- function(c, e, p){  
  OverallChange <- c*p*(1-p)-e*p  
  return(OverallChange)  
}
```

Let's see if your function works. Try running your function below:

```
RateOfChange(c=0.4, e=0.2, p=0.4)
```

```
## [1] 0.016
```

```
#Answer should be 0.016
```

```
# Try it for a few different parameter values and check it seems as expected...
```

```
RateOfChange(c=0.4, e=0.1, p=0.7)
```

```
## [1] 0.014
```

Make sure the function and parameter names are all correct!

Step-by-step simulation

After creating the function for calculating the rate of change, the next function we need to create is a function that progresses the model by one time step. Essentially, our model works by calculating the rate of change with the current patch occupancy, then updating the patch occupancy according to the rate of change. We will then use the same function calculate the rate of change again with the new patch occupancy and repeat the process many times to track patch occupancy over time.

When modelling using differential equations, we need to progress at a very small step in order to get accurate results. Here is an analogy: imagine a car driving around a city; you know the position of the car at the start of its journey and you want to find out where it is at the end of its journey. But the only information you have to help you find out is its speed and direction of movement. If you record its speed and direction every hour, you will not have any idea where the car ends up because it could have changed speed and direction many times during that hour and you will not have included this information. If you record the speed and direction every second, you will be able to deduce the route of the car almost exactly and will have a very accurate view of where it ends its journey.

In our models, the formula we have evaluated gives the speed of movement (and the sense of direction comes from whether the number is positive or negative). We are trying to track a very small step (Δt) so we can see clearly what the model predicts with minimal errors.

Recall that the rate of change equation of p is written as $\frac{dp}{dt}$ and that we have an equation (and now an R function!) to calculate this rate of change (based on the values e , c , and p). Remember also that this notation refers to an infinitely small amount of time (dt), and the rate of change tells us how p changes over this time.

When we translate this into a computational problem, instead of thinking about how p changes over an *infinitely small* amount of time (dt), we will instead consider how p changes over a *very small* (but not infinitely small) amount of time, Δt . You can read this Δt as “change in t ”. We can then use equation below (the **discrete** version of the differential equation above) to estimate the change in p (Δp):

$$\Delta p = \Delta t \times \frac{dp}{dt}$$

And so the value of p after time Δt has passed can be estimated by:

$$p + \Delta t \times \frac{dp}{dt}$$

In the Levins model, this would represent the patch occupancy we had before, p , plus the small change expected, Δp (which may be positive or negative). From this equation, as long as we know p , e and c , we can calculate the patch occupancy a tiny way (Δt) into the future.

Throughout the EoE practicals, we will term this small step into the future a *time step*, and we will denote it by dt . By multiplying the rate of change ($\frac{dp}{dt}$) by dt , we obtain an approximation of the small change in p which occurs during one time step, which we will call dp . **This is a general approach to approximating differential equations so that we can simulate them using code** and is a good approximation in many situations. The method is more and more accurate the smaller we make the time step dt . This method is one of the simplest ways to get a numerical solution to a differential equation, and is called the Euler method. If you are interested in differential equations, there are many more complex and efficient numerical solutions out there; this is an active area of research in mathematics and computational science.

We can write a function to compute the change in p over a short time step dt by including dt as the following code shows:


```
## DO NOT EDIT ##
```

```
ChangeP <- function(c, e, p, dt=0.01){  
  # Returns the small change in patch occupancy which occurs across one time  
  # step of size dt.  
  
  output <- RateOfChange(c, e, p)  
  # Calculating the rate of change from your previous function  
  output <- output*dt # Multiplying it by the time step  
  
  return(output)  
}
```

Question 1.2

With this mathematical issue taken care of, let's use the function above to actually start modelling. To start, we need to create another function (named `LevinsStep`) that progresses the model for one time step. The function should do 3 things:

- Calculate the small change in patch occupancy using the `ChangeP` function above
- Add the small change to the original patch occupancy
- Output the new patch occupancy (using `return` in the function)

```
### QUESTION 1.2 ###
```

```
LevinsStep <- function(c, e, p, dt=0.01){  
  
  # Let's first calculate the change in patch occupancy  
  Change <- ChangeP(c, e, p, dt)  
  
  # Finish off the function here (calculate the new patch occupancy and output it):  
  p <- p + Change  
  return(p)  
}
```

Test your function to make sure no errors pop up:

```
LevinsStep(c=0.2, e=0.1, p=0.7)
```

```
## [1] 0.69972
```

By being able to progress the model by one time step and updating the patch occupancy, we will be able to keep running the `LevinsStep` function over and over again to model the changes of patch occupancy over time.

Question 1.3

What we need now is a function which runs a loop that calculates the change in patch occupancy for a set number of times, then records the output in a vector.

Here are the requirements:

- The function should be named `Loop`
- The function must have 4 input parameters:
 - Colonisation
 - Extirpation
 - Patch
 - Times
- The loop repeats a set number of times, defined by the parameter `Times`
- The function must output the vector of patch occupancies after each time step
- The default time step `dt=0.01` should be used

```
### QUESTION 1.3 ###
```

```
Loop <- function(c, e, p, Times){  
  PatchOccupancy <- c() #create an empty vector for recording patch occupancy  
  
  for (i in 1:Times){  
    p <- LevinsStep(c,e,p,dt=0.01) #calculate new patch occupancy using LevinsStep  
    PatchOccupancy <- c(PatchOccupancy, p) #record  
    PatchOccupancy # the output in the vector.  
  }  
  
  PatchOccupancy #Output the final list  
}
```

Let's see if your function works correctly:

```
occupancy_over_time <- Loop(c=0.2,e=0.1,p=0.7,Times=10)  
print(occupancy_over_time)
```

```
## [1] 0.6997200 0.6994405 0.6991615 0.6988830 0.6986050 0.6983275 0.6980505  
## [8] 0.6977740 0.6974980 0.6972225
```

The patch occupancy shouldn't change very much since we are only progressing through a few small time steps.

Running the model

Now that you have a list that records the change in patch occupancy over time using the equation, we basically completed our model! Well done - you've already written your first ecological simulation model. Your final step is to plot the results.

First, let's produce some results:

```
# This produces the output vector called Data, with the model running for 5000 time steps  
# The initial patch occupancy was 0.8, and the colonisation rate and extirpation rate  
# were 0.2 and 0.1 respectively.
```

```
## DO NOT EDIT ##
```

```
Data <- Loop(c=0.2, e=0.1, p=0.8, Times=5000)
```

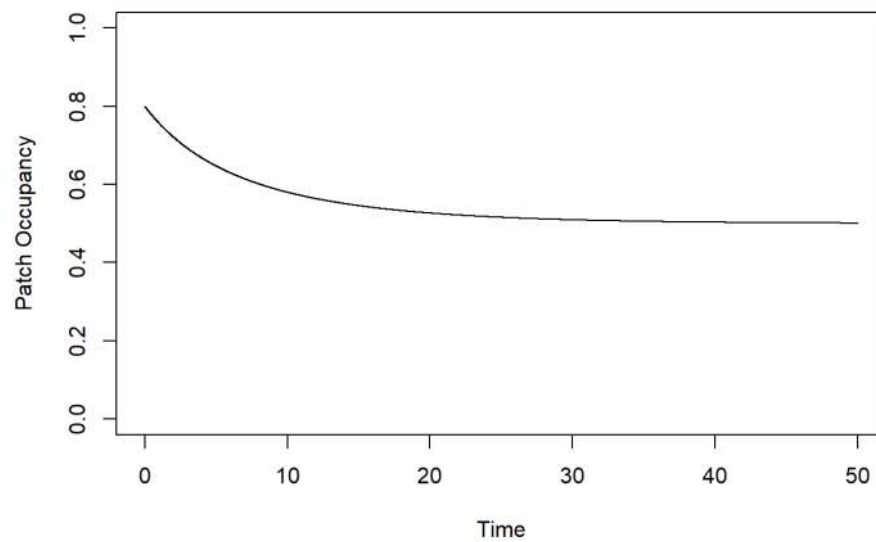
Question 1.4

In the code box below, plot a line graph (named Figure 1.4) that tracks the change in patch occupancy over time. You should use the results generated above (named `Data`) as the data input. Modify the y-axis so that the range is from 0 to 1 (using the `ylim` argument in the `plot` function). The values for the x-axis (time) are `1:5000` divided by 100, since each time step was of size 0.01.

This part will be visually marked! so make sure your graph has all the features you would expect of a graph in a project or published paper. Refer to the plots above to see what kinds of features you should include.

```
# write your code here to plot a figure  
time <- seq(0.01,50,by=0.01)  
plot(time,Data,xlab="Time",ylab="Patch Occupancy",main="Figure 1.4: The change in patch occupancy over time",ylim=c(0,1),type="l")
```

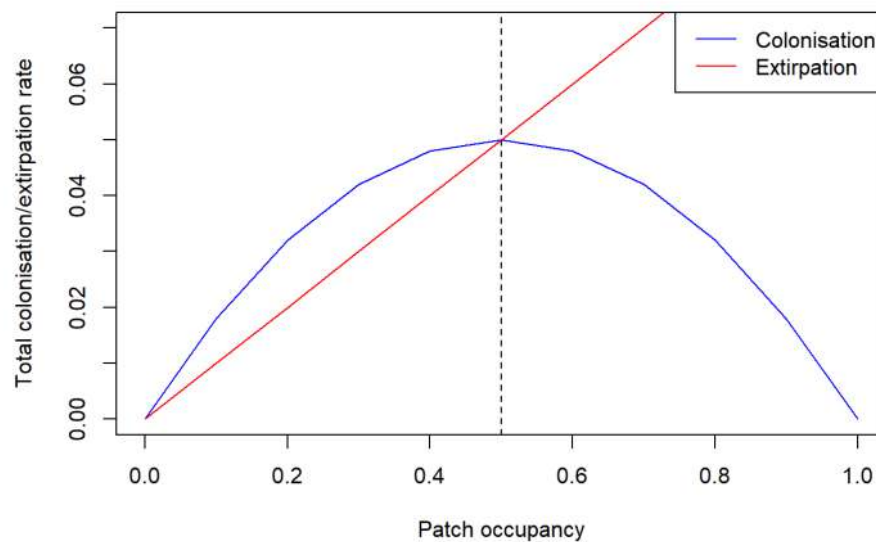

Figure 1.4: The change in patch occupancy over time



You can now see how the patch occupancy changes over time.

You should see the line decrease and stabilise at an equilibrium value. In fact, you can visualise the equilibrium by combining the extirpation and colonisation plots (Figures 1.2 and 1.3 from the introduction):

```
p <- seq(0,1,by=0.1)
c <- 0.2
e <- 0.1
Col <- c*p*(1-p)
Ext <- e*p
plot(p,Col,type="l",col="blue",xlab="Patch occupancy",
      ylab="Total colonisation/extirpation rate", ylim=c(0,0.07))
lines(p,Ext, col="red")
legend("topright",legend=c("Colonisation","Extirpation"), col=c("blue","red"), lty=1)
abline(v=0.5, lty=2)
```



To the left of the dashed line, colonisation is higher than extirpation, hence patch occupancy will increase. To the right of the dashed line, extirpation is higher than colonisation, hence patch occupancy will decrease. As a result, the population will stabilise at the intersection point between the two lines! (In our case, it is patch occupancy=0.5, as seen from the simulation output plot you produced.) Remember MacArthur and Wilson's theory of island biogeography from the lectures where immigration and extinction rates cross to predict how many species there are on an island? Well, it's a similar principle here to predict how many patches your species occupies.

Question 1.5

We can also calculate where extirpation and colonisation are balanced from the equation. You are looking for a case where the rate of change is 0, meaning that $\frac{dp}{dt}$ will not change (i.e. the car is parked in our earlier analogy).

$$\frac{dp}{dt} = 0 = cp(1-p) - ep$$

Do the algebra yourself and write your answer and workings here (your first line should be the equation $cp(1-p) - ep = 0$). You are looking to find out what value of p satisfies this equation. Then, plug in our previous values ($c=0.2$), ($e=0.1$) and verify that the equilibrium value ($p=0.5$) satisfies this equation.

$$cp(1-p) - ep = 0 \quad | \quad 0.2p(1-p) - 0.1p = 0 \quad | \quad 0.2p - 0.2p^2 - 0.1p = 0 \quad | \quad 0.2p^2 - 0.1p = 0 \quad | \quad p(0.2p - 0.1) = 0 \quad | \quad 0.2p - 0.1 = 0 \quad | \quad p = 0.1/0.2 \quad | \quad p = 0.5$$

You can now play with the parameters of the model using the function and plotting code you produced and see when the population crashes and when it survives, and what value p approaches at equilibrium. You can do this to make sure you got the right answer in your algebra above. It's not for marks, but it's worth doing to check your own work is correct!

```
dy_dx <- diff(Data) / diff(time) # Calculate the numerical derivative
```

```
zero_gradient_Data <- Data[which(abs(dy_dx) < 0.00013)] # Find points where the gradient is close to zero
```

```
zero_gradient_Data
```

Try changing the parameters of the model and see how the results change:

```
Data <- Loop(c=0.2,e=0.1,p=0.8,Times=5000)
```

```
time <- seq(0.01,50,by=0.01)
```

```
plot(time,Data,xlab="Time",ylab="Patch Occupancy",main="Figure 1.5: The change in patch occupancy over time",ylim=c(0,1),type="l")
```

```
dy_dx <- diff(Data) / diff(time) # Calculate the numerical derivative
```

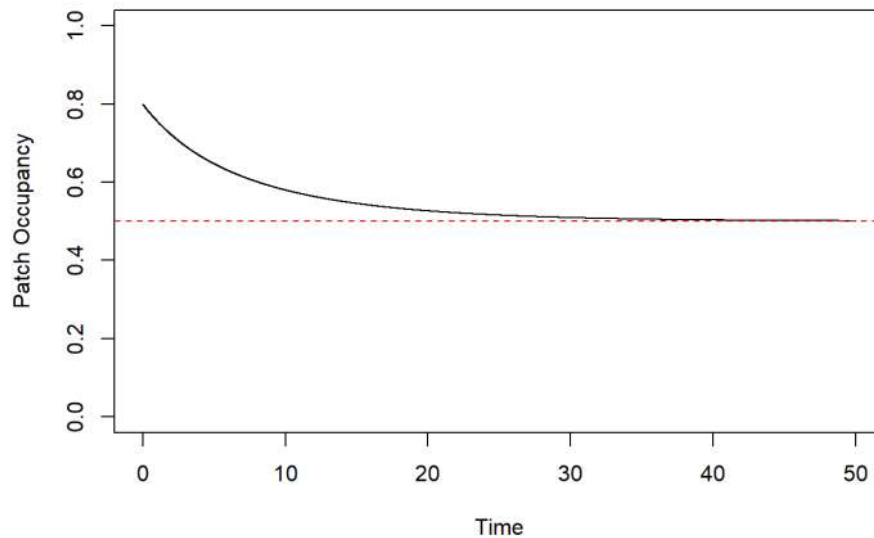
```
zero_gradient_Data <- Data[which(abs(dy_dx) < 0.00013)] # Find points where the gradient is close to zero
```

```
zero_gradient_Data
```

```
## [1] 0.5012962 0.5012949 0.5012936 0.5012923 0.5012910 0.5012897 0.5012884
## [8] 0.5012871 0.5012858 0.5012845 0.5012832 0.5012819 0.5012807 0.5012794
## [15] 0.5012781 0.5012768 0.5012755 0.5012743 0.5012730 0.5012717 0.5012704
## [22] 0.5012691 0.5012679 0.5012666 0.5012653 0.5012641
```

```
abline(h = zero_gradient_Data, col = 'red', lty = "dashed") # Adding a dashed horizontal line at y = 0, where you are expecting to see the equilibrium value
```

Figure 1.5: The change in patch occupancy over time



Challenge questions - Stochasticity

You may remember the concept of stochasticity from the lectures on dispersal and ecological drift. Now we're going to see what it does to the Levins metapopulation model.

One way to introduce stochasticity to your model is to generate a number following normal distribution with a defined mean and variance.

Challenge Question 1.1

Let's try doing this! In the code input below, write a line of code that creates a random number based on a normal distribution (mean=5, sd=1). **Tip: Google is helpful when learning new R commands and functions!** If you get stuck, look up the function `morm`.

```
### CHALLENGE QUESTION 1.1 ###

random_number <- morm(1, mean = 5, sd = 1)
```

Challenge Question 1.2

Now that you know how to produce random numbers, you can add stochasticity into the models. Repeat what you did above in `Loop`, but now each time step, draw from a normal distribution to give the colonisation (c) and extirpation (e) rates that apply, instead of keeping them fixed. Recall that c and e should be between 0 and 1, as they are probabilities. You will probably need to experiment with the variance of this distribution to see interesting results. This is an open question so add whatever you like in the below cells.

```
### CHALLENGE QUESTION 1.2 ###

Loop2 <- function(mean_value = 0.5, sd_value = 0.2, initial_p_values) {
  # Find the overall range of time
  time_range <- range(time)
```

```

time_range <- range(time)

# Set up the plot with x-axis limits
plot(1, type = "n", xlab = "Time", ylab = "PatchOccupancy", main = "PatchOccupancy over Time", ylim = c(0, 1), xlim = time_range)

# Create an empty list to store the results
results_list <- list()

for (initial_p in initial_p_values) {
  PatchOccupancy <- numeric(length(time))
  p <- initial_p

  for (i in seq_along(time)) {
    # generate random c and e values between 0 and 1 from a normal distribution
    c <- pmin(pmax(morm(1, mean = mean_value, sd = sd_value), 0), 1)
    e <- pmin(pmax(morm(1, mean = mean_value, sd = sd_value), 0), 1)

    p <- LevinsStep(c, e, p, dt = 0.01)
    PatchOccupancy[i] <- p
  }

  # Plot the line for each set of initial_p values
  lines(time, PatchOccupancy, col = rainbow(length(initial_p_values))[match(initial_p, initial_p_values)])

  # Store results in a data frame
  results_df <- data.frame(Time = time, PatchOccupancy = PatchOccupancy, Initial_p = as.character(initial_p))

  # Append the results to the list
  results_list <- c(results_list, list(results_df))
}

# Combine the data frames from the list
results <- do.call(rbind, results_list)
legend("topright", legend = as.character(initial_p_values), col = rainbow(length(initial_p_values)), lty = 1, title = "Initial_p")
}

# Call the function with multiple initial_p values
Loop2(initial_p_values = c(0.2, 0.5, 0.8))

# two curves will be shown on a single graph, which makes comparison easier

# the PatchOccupancy value is generated randomly each turn, therefore there are small fluctuations on the graph

# The graph shows that higher initial p values will have faster rate of decrease of PatchOccupancy over time, and they both finally approach to an asymptote.

## I did another version by using ggplot

Loop2 <- function(mean_value = 0.5, sd_value = 0.2, initial_p_values) {
  results_list <- list() # Create an empty list to store the results

  for (initial_p in initial_p_values) {
    PatchOccupancy <- c()
    TimeVector <- time
    p <- initial_p

    for (i in seq_along(time)) { #Generates a sequence from 1 to the length of vector time

      #generate random c and e values between 0 and 1 from a normal distribution
      c <- pmin(pmax(morm(1, mean = mean_value, sd = sd_value), 0), 1)
      e <- pmin(pmax(morm(1, mean = mean_value, sd = sd_value), 0), 1)

      p <- LevinsStep(c, e, p, dt = 0.01)
      PatchOccupancy <- c(PatchOccupancy, p)
    }

    # Store results in a data frame
    results_df <- data.frame(Time = TimeVector, PatchOccupancy = PatchOccupancy, Initial_p = as.character(initial_p))
  }
}

```

```

results_df <- data.frame(time = timevector, PatchOccupancy = PatchOccupancy, initial_p = as.character(initial_p))

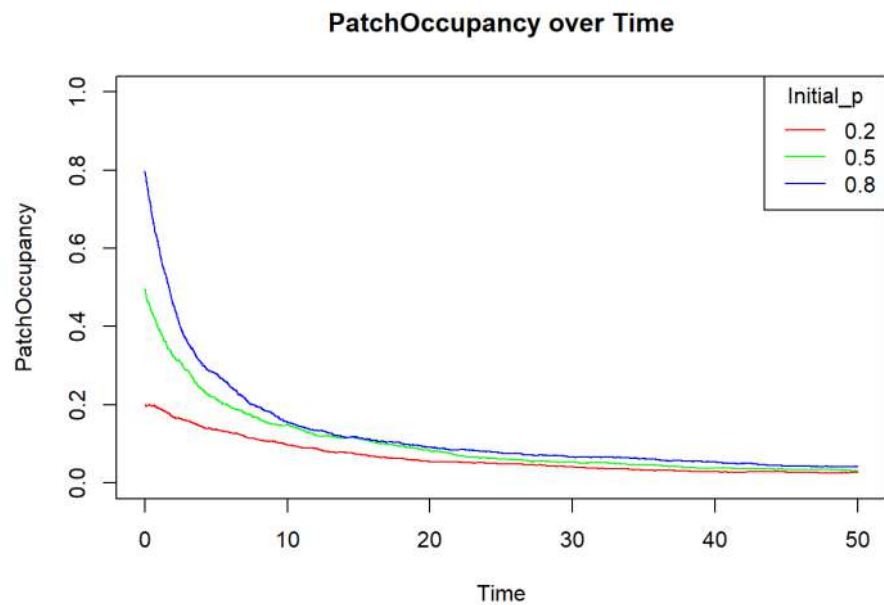
# Append the results to the list
results_list <- c(results_list, list(results_df))
}

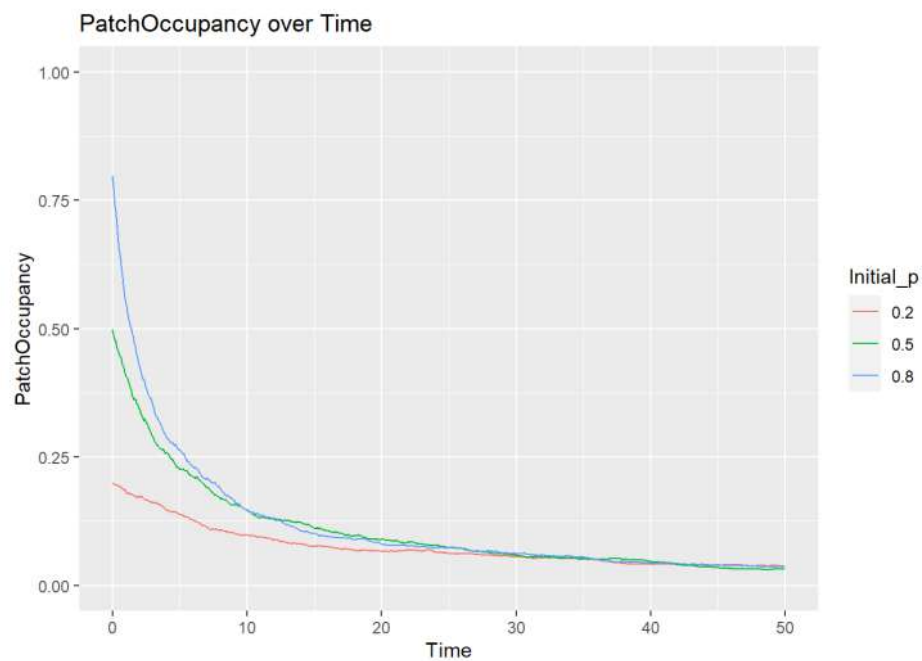
# Combine the data frames from the list
results <- do.call(rbind, results_list) #execute function rbind to combine the objects in the list

# Plot the graph using ggplot2
library(ggplot2)
ggplot(results, aes(x = Time, y = PatchOccupancy, color = Initial_p)) + geom_line() + labs(x = "Time", y = "PatchOccupancy") + ggtitle(
  "PatchOccupancy over Time") + coord_cartesian(ylim = c(0, 1)) # Set y-axis limits
}

# Call the function with multiple initial_p values
Loop2(initial_p_values = c(0.2, 0.5, 0.8))

```





Congrats on completing the first notebook! After this, you should be well equipped to produce more interesting and complex models in the upcoming RMarkdown documents.

Let have a quick summary of what we have learnt and practiced so far:

1. Writing functions;
2. Using a for loop;
3. Running a simulation over time and plotting the results;
4. (Challenge) Introducing stochasticity.

With what you learned, you can basically produce most mechanistic models out there. In the next workbook, we will move on to modelling fisheries, introducing more interesting dynamics; and we can even apply it onto fishery management. See you there!

----- **End of document** -----

FINAL GRADE

91 / 100

GENERAL COMMENTS

Question 1.1 (out of 10)

10
excellent

Question 1.2 (out of 10)

10
excellent

Question 1.3 (out of 10)

10
excellent

Question 1.4 (out of 10)

10
excellent

Question 1.5 (out of 10)

6
correct final answer but should have solved tl

Total for main questions (out of 70)

64.4

Total for challenge questions (out of 15)

13.5
brilliant answer - great to attempt graph with

Quality of graphical outputs (out of 5)

5
excellent

Quality of code (out of 5)

4
good but some room for improvement on co

final total

91

PAGE 1

PAGE 2

PAGE 3

PAGE 4

PAGE 5

PAGE 6

PAGE 7

PAGE 8

PAGE 9

PAGE 10

PAGE 11

PAGE 12

PAGE 13

2-Harvesting-v4.html

by Yuki Wu

Submission date: 08-Feb-2024 09:21PM (UTC+0000)

Submission ID: 224042949

File name: 2-Harvesting-v4.html (1.04M)

Word count: 4377

Character count: 22084

2: Harvesting and fisheries

Essentials of Ecology coding coursework

Yuki Wu

February 2024

- Introduction
- Model specification
- Building the model
 - **Question 2.1**
 - **Question 2.2**
 - **Question 2.3**
 - **Question 2.4**
 - **Question 2.5**
- Fishery management
 - Creating a fixed catch model
 - **Question 2.6**
 - **Question 2.7**
 - Finding the MSY
 - **Question 2.8**
 - **Question 2.9**
 - Challenge: Creating a fixed effort harvest model
 - **Challenge Question 2.2**

Introduction

Welcome to the second workbook! We will do some classic fishery modelling, to see if we can use models in R to make some simple management decisions.

Model specification

We will use one of the most basic models for population growth: the logistic growth model, or density dependent growth. Basically, this means that the population will grow exponentially when small. As the population size approaches a certain carrying capacity, its growth will slow. If the population is greater than the carrying capacity, it will decrease.

The principle is the same as you saw in the lecture on models where we looked at a model of a whale population. The main difference here is that our equation gives you the growth rate at any point in time, so you could update it as often as you like. In contrast, the whale population formula in lecture 2 gave us next year's population size as a function of this year's population size - so it was always jumping forward by a fixed amount of time: one year.

Here is the logistic growth equation:

$\frac{dN}{dt} = rN \left(1 - \frac{N}{K} \right)$ where:

- r = intrinsic growth rate
- N = population size
- K = carrying capacity

Here is what the logistic growth model looks like:

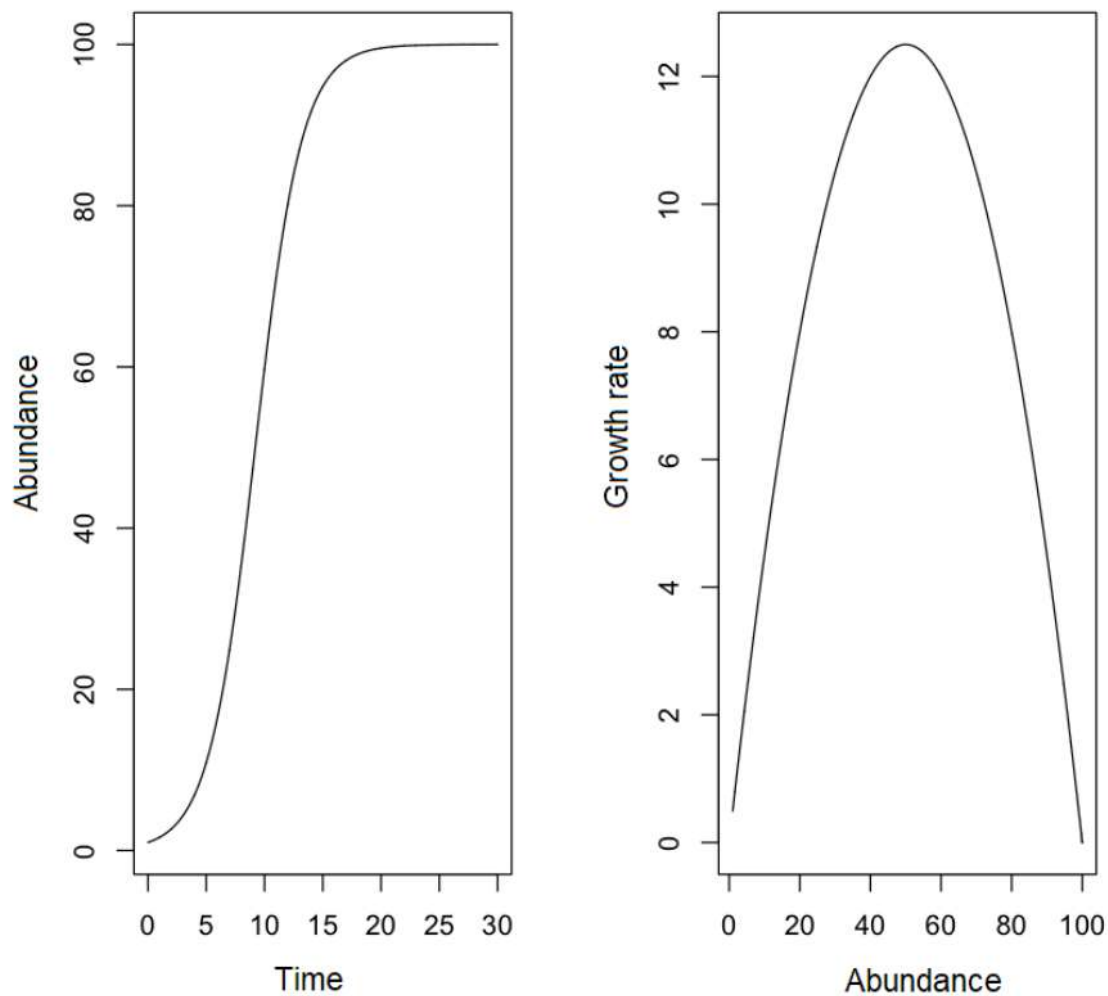


Figure 2.1: Behaviour of a population under the logistic growth model.

A few things to note in Figure 2.1 above:

- (Figure 2.1A) Fish abundance increases exponentially at the start but slows down as it approaches the carrying capacity, where growth finally stops.
- (Figure 2.1B) When plotting the growth rate against abundance, we can see that growth rate is largest when the population size is at half of its carrying capacity.

Building the model

Lets start building a model from scratch again. The ultimate goal for this model is to produce two output graphs similar to the ones shown in Figure 2.1. One of the graphs should show the change in abundance through time, and the second one to show how the population growth rate changes by changing abundance.

This is very similar to the model you built in the last workbook, so it may help to refer back to it as you go.

Question 2.1

Firstly, create a function (named `RateOfChange`) that takes the model parameters and current fish abundance as inputs and returns us the rate of population change ($\frac{dN}{dt}$), just as you did in the last worksheet.

Requirements:

- Input:
 - Intrinsic growth rate (r), named `r`
 - Carrying capacity (K), named `K`
 - Fish abundance (N), named `N`
- Output:
 - The rate of change ($\frac{dN}{dt}$)

Question 2.1

```
RateOfChange2 <- function(r,K,N){
  dN_dt <- r*N*(1-N/K)
  return(dN_dt)
}
```

Check if your function works correctly:

```
RateOfChange2(r=2,K=100,N=50)
```

```
## [1] 50
```

The answer should be 50

Question 2.2

Next, we need to write a function (named `LogisticStep`) to progress the population by a very small amount of time given by step size `dt`. The function should use the function you just wrote to get the rate of change, then calculate and output the population abundance at the next time step. The inputs will be `r`, `K`, `N`, and `dt`, and the output will be the new value of `N`. As you may remember from the last notebook, we need to multiply the rate of change by `dt`, so that we are progressing one very small step in time. Remember to include `dt` in your function!

Note that in the last workshop, you did this in two steps (`ChangeP` and `LevinsStep`).

Question 2.2

```
LogisticStep <- function(r,K,N,dt=0.01){
  change <- RateOfChange2(r,K,N)
  change <- change*dt
  N <- N + change
  return(N)
}
```

Check if your function is correct:

```
LogisticStep(r=0.5, K=100, N=50, dt=0.01)
```

```
## [1] 50.125
```

The answer should be 50.125

Question 2.3

Next, you will write a function (named `Loop`) which runs a while loop for a fixed Time from 0 to `tEnd` increasing Time by step within each cycle of the loop. So, `tEnd` should be another input to your function just as it was in the last worksheet. You should calculate the change in population abundance and record population abundance into a vector `NVector` with every cycle of the loop. You will also need to record the corresponding times in a vector `TimeVector` just as before with every cycle of the loop.

There are a couple of added things your function should do as well. We want to plot two different graphs, so we also need to calculate (using the `RateOfChange` function you created) and record the rate of change ($\frac{dN}{dt}$) and record that in a vector `RateVector` with each cycle of the loop.

The input of your function `Loop` should be 5 parameters:

- Intrinsic growth rate (named `r`)
- Carrying capacity (named `K`)

- Initial population abundance (named `NStart`)
- Amount of time to run the simulation for (named `tEnd`)
- Small time step for each cycle of the loop (`dt`) with a default value of 0.01

A function can only return one thing, but you need your function to return three vectors (`NVector`, `RateVector` and `TimeVector`). To do this, at the end of the function you'll need to combine those vectors into a data frame with three columns using the `data.frame` function, you can then return the data frame which now contains all your data.

```
### QUESTION 2.3 ###

Loop3 <- function(r, K, NStart, tEnd, dt = 0.01) {
  NVector <- c()
  RateVector <- c()
  TimeVector <- c()
  Time <- 0

  while (Time <= tEnd) {
    NVector <- c(NVector, NStart)
    NStart <- LogisticStep(r, K, NStart, dt = dt)
    RateVector <- c(RateVector, RateOfChange2(r, K, NStart))
    Time <- Time + dt
    TimeVector <- c(TimeVector, Time)
  }

  # Combine vectors into a data frame
  Output <- data.frame(NVector, RateVector, TimeVector)

  return(Output)
}
```

Check your answer!

```
SimulationResults <- Loop3(r=0.5, K=100, NStart=1, tEnd=30)
# How big is the data frame?
dim(SimulationResults)
```

```
## [1] 3000 3
```

```
# Let's look at the first ten rows.
SimulationResults[1:10,]
```

```
##   NVector RateVector TimeVector
## 1 1.000000 0.4974254    0.01
## 2 1.004950 0.4998624    0.02
## 3 1.009924 0.5023111    0.03
## 4 1.014923 0.5047715    0.04
## 5 1.019946 0.5072438    0.05
## 6 1.024994 0.5097279    0.06
## 7 1.030066 0.5122239    0.07
## 8 1.035163 0.5147319    0.08
## 9 1.040286 0.5172518    0.09
## 10 1.045433 0.5197839    0.10
```

Before we continue working through our fishery model, there is a feature we should add to our model to improve the speed of processing our model results. Did you notice the small wait when R is processing the model results? This is caused because your computer is saving a lot more data from the simulation than it really needs to plot the graph at the end.

In our function `Loop`, we are recording data into an output vector at every time step. This recording takes a lot of time, likely more time even the calculations themselves. What we can do is basically NOT record the data at every step, but to record let's say once every 100 time steps of the simulation, but still make the simulation progress one small step at a time. This can significantly cut down on the processing time because R does not have to store too much information. (Have a look at the nested loops examples in the Foundations worksheet).

Question 2.4

Let's try adding an inner loop into our `Loop` function. In your previous `Loop` function, there should be two main types of code. One is to calculate the rate of change and new abundance values. The second should be to record the data into vectors. All you need to do is to add an extra for loop with curly brackets surrounding the first type of code, which calculates new values. This for loop should cycle through 100 time steps of the simulation, but will not contain the vector-writing piece of code. By doing that, the inner loop progress the simulation 100 steps each time the outer loop goes around once, however, the data will be recorded only once for each time the outer while loop cycles round. Call this function `LogisticSimulator` and copy your code down from `Loop` above, then make the necessary edits, to make this function.

```
### QUESTION 2.4 ###

LogisticSimulator <- function(r, K, NStart, tEnd, dt = 0.01) {
  NVector <- c()
  RateVector <- c()
  TimeVector <- c()
  Time <- 0

  while (Time <= tEnd) {
    # Inner loop for progressing the simulation by 100 time steps
    for (i in 1:100) {
      NStart <- LogisticStep(r, K, NStart, dt = dt)
      Rate <- RateOfChange2(r, K, NStart)
      Time <- Time + dt
    }

    # Record data after the inner loop
    NVector <- c(NVector, NStart)
    RateVector <- c(RateVector, Rate)
    TimeVector <- c(TimeVector, Time)
  }

  # Combine vectors into a data frame
  Output <- data.frame(NVector, RateVector, TimeVector)

  return(Output)
}
```

Check your answer now:

```
SimulationResults <- LogisticSimulator(r=0.5, K=100, NStart=1, tEnd=30)
print(SimulationResults)
```

```
##      NVector  RateVector TimeVector
## 1  1.636140  0.804685249      1
## 2  2.666109  1.297513855      2
## 3  4.316141  2.064925179      3
## 4  6.915151  3.218478724      4
## 5 10.901669  4.856602431      5
## 6 16.773979  6.980157495      6
## 7 24.927853  9.356937116      7
## 8 35.366581 11.429315176      8
## 9 47.422737 12.466788578      9
##10 59.794472 12.020341633     10
##11 71.039386 10.286721120     11
##12 80.185738  7.944106240     12
##13 86.975447  5.664081697     13
##14 91.681045  3.813452301     14
##15 94.788922  2.469762473     15
##16 96.776747  1.559679718     16
##17 98.022194  0.969344608     17
##18 98.792441  0.596488336     18
##19 99.264980  0.364808651     19
##20 99.553446  0.222279808     20
##21 99.729012  0.135127071     21
##22 99.835667  0.082031601     22
##23 99.900387  0.049756930     23
##24 99.939634  0.030165024     24
##25 99.963423  0.018281802     25
##26 99.977840  0.011077778     26
##27 99.986575  0.006711766     27
##28 99.991867  0.004066221     28
##29 99.995073  0.002463355     29
##30 99.997015  0.001492286     30
```

This should give the same pattern, but with far fewer data points. For example, the `TimeVector` portion of the data frame should look like "1, 2, 3, 4, ..." rather than "0.01, 0.02, 0.03, 0.04, ...".

Question 2.5

The last part of our model is to create the output plots. As we outlined at the start of this worksheet, we would like to plot two output graphs, similar to Figure 2.1. To do this, write two functions `PlotN` and `PlotRate`. Both functions should receive as an input the data frame created by the `LogisticSimulator` function above and plot the appropriate graph.

Graphs required:

1. Change in abundance through time: abundance as y-axis and time as x-axis (function named `PlotN`)
2. Rate of change curve: rate of change (growth rate) as y-axis and population abundance as x-axis (function named `PlotRate`)

Remember to add in axis labels as well!

```
#### QUESTION 2.5

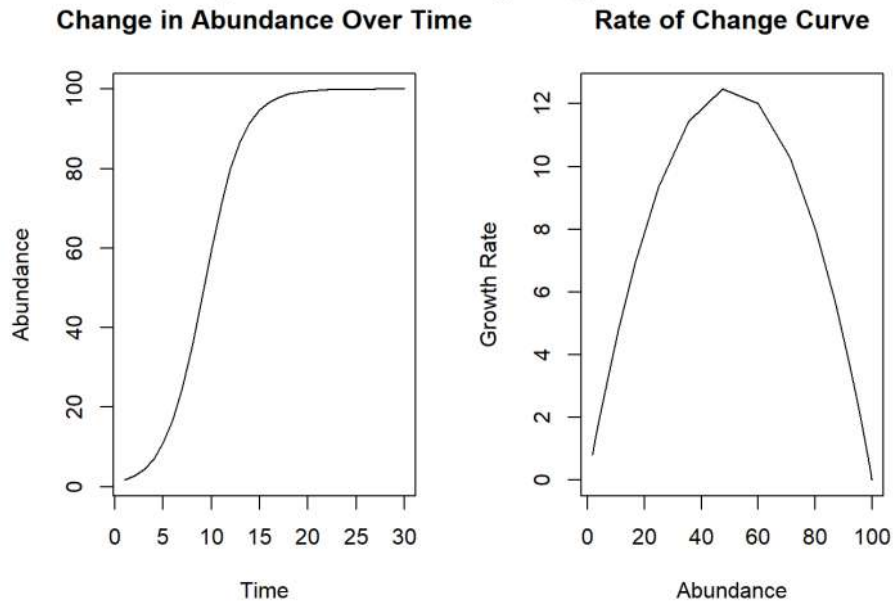
# Function to plot abundance over time
PlotN <- function(SimulationResults) {
  plot(SimulationResults$TimeVector, SimulationResults$NVector, type = "l", xlab = "Time", ylab = "Abundance", main = "Change in Abundance Over Time")
}

# Function to plot rate of change curve
PlotRate <- function(SimulationResults) {
  plot(SimulationResults$NVector, SimulationResults$RateVector, type = "l", xlab = "Abundance", ylab = "Growth Rate", main = "Rate of Change Curve")
}
```

Now that we have all the functions needed for the model, we can construct the final model! Run the code below to see the model output.


```
## DO NOT EDIT ##
df <- LogisticSimulator(r=0.5, K=100, NStart=1, tEnd=30)
par(mfrow=c(1,2)) # make a two panel plot of the next two graphs
PlotN(df)
PlotRate(df)
title("Figure 2.2: Simulated logistic growth model",line=-1,outer=TRUE)
```

Figure 2.2: Simulated logistic growth model



Fishery management

An important concept in fishery management is the **maximum sustainable yield (MSY)**. MSY is the maximum natural growth rate of the population; the fish that you can take sustainably because they are able to be replenished by population growth at the same rate.

The first harvesting model we will simulate is the **fixed-quota harvesting** model, which is a strategy where a number is set for how much fish can be caught.

It can be visualised as follows:

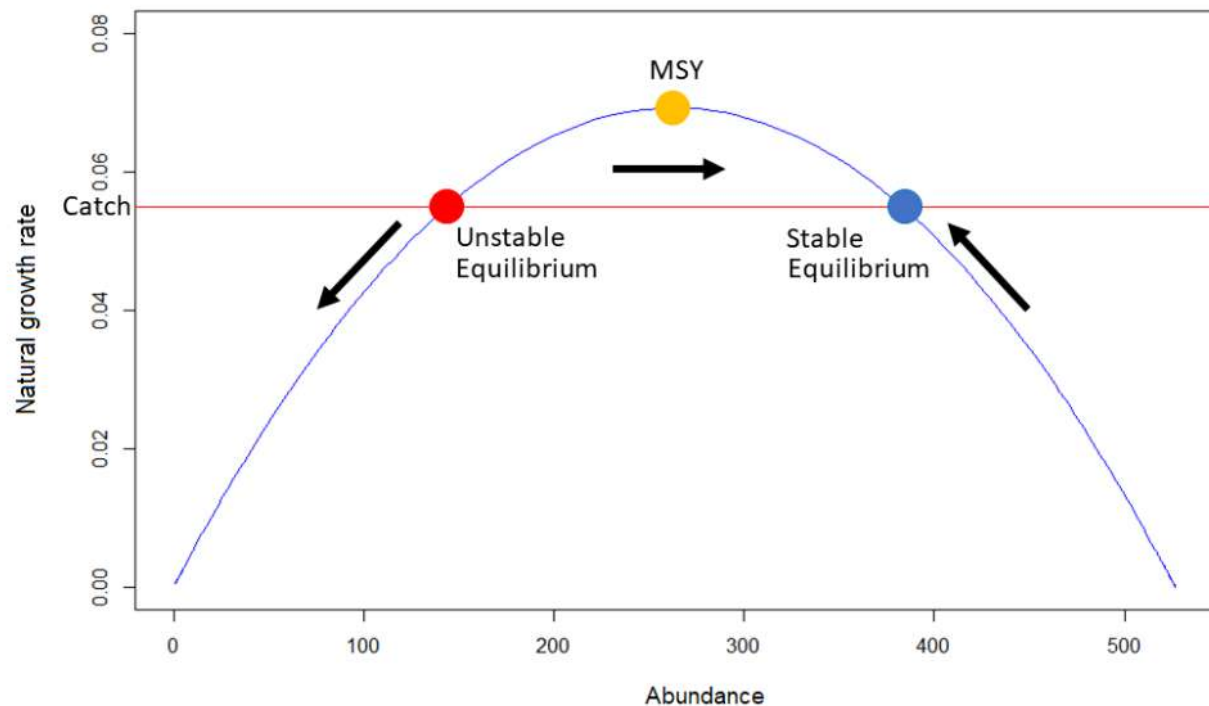


Figure 2.3: Behaviour of the logistic growth model with the addition of fixed-quota harvesting. Arrows represent the trajectory of the population.

Here is how we interpret Figure 2.3:

- The red line represents the catch quota, or the mortality of the population due to harvesting
- When the natural growth rate is higher than the catch quota, fish abundance increases, hence it moves to the right
- When the catch quota is higher than natural growth rate, abundance decreases, hence the curve moves to the left

As we can see, if a fixed quota was set for catch, there will be a stable and an unstable equilibrium. The stable equilibrium you have seen before, in this case if the population gets large, the natural growth rate will fall and population will decrease back to the equilibrium point, conversely if the population falls below equilibrium it will increase and get back to equilibrium again. The unstable equilibrium is different to what you've seen before - in theory the natural growth and harvest are perfectly balanced, but a tiny decrease from that point and the movement is away from equilibrium so the population crashes to extinction. You can think of an unstable equilibrium as a pen balanced on its end - it could be done in theory but in practice the tiniest gust of wind will blow it over.

The maximum sustainable yield (MSY) is the maximum value of the fixed catch quota which is theoretically possible while maintaining a viable population. You can imagine raising the red "catch" line up and observing that it is possible to increase the catch quota up until the MSY point. Think about what happens to the stable and unstable equilibria as you raise the value of the catch quota.

Creating a fixed catch model

Now that you understand the concepts behind fishery modeling and fixed quota harvesting, it is time to model it! The formula for rate of population growth is now given by:

$\frac{dN}{dt} = rN \left(1 - \frac{N}{K}\right) - C$ where:

- r = intrinsic growth rate
- K = carrying capacity
- C = fixed catch quota
- N = population size

Question 2.6

Update your previous `RateOfChange` and `LogisticStep` functions (calling them `RateOfChangeCatch` and `CatchStep`) with an extra parameter called `C` (catch quota) added. You can copy and paste the previous functions and modify it to create these new ones.

```
### QUESTION 2.6
```

```
RateOfChangeCatch <- function(r,K,C,N){  
  dN_dt <- r*N*(1-N/K)-C  
  return(dN_dt)  
}  
  
CatchStep <- function(r,K,C,N,dt){  
  change <- RateOfChangeCatch(r,K,C,N)  
  N <- N+dt*change  
  return(N)  
}
```

Let's see if it works correctly:

```
RateOfChangeCatch(r=0.5, N=50, K=100, C=10)
```

```
## [1] 2.5
```

```
# Answer should be 2.5
```

```
CatchStep(r=0.5, N=50, K=100, C=10, dt=0.01)
```

```
## [1] 50.025
```

```
# Answer should be 50.025
```

Question 2.7

Now, we need to modify the `LogisticSimulator` function a little bit to include the catch parameter `C`. Do this in the box below, copying down what you already have and then modifying it. Name the new function `CatchSimulator`.

Your `RateVector` should be the total growth rate (the **actual** rate of change of abundance $\frac{dN}{dt}$), using `RateOfChangeCatch`, rather than the natural growth rate. This total growth rate becomes the aggregate of the natural growth rate and the mortality due to catching.

QUESTION 2.7

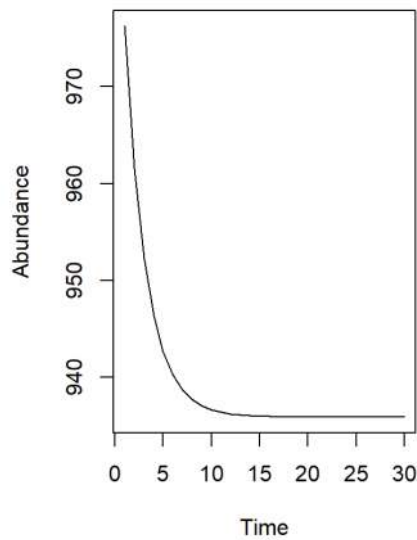
```
CatchSimulator <- function(r, K, C, NStart, tEnd, dt = 0.01) {  
  NVector <- c() # Initialize population abundance vector  
  TimeVector <- c() # Initialize time vector  
  RateVector <- c() # Initialize rate of change vector  
  time <- 0 # Initialize time  
  
  while (time <= tEnd) {  
    for (i in 1:100){  
      N <- CatchStep(r, K, C, NStart, dt)  
      ratechange <- RateOfChangeCatch(r, K, C, NStart)  
      NStart <- N  
      time <- time + dt  
    }  
  
    NVector <- c(NVector, NStart)  
    TimeVector <- c(TimeVector, time)  
    RateVector <- c(RateVector, ratechange)  
  }  
  
  MyData <- data.frame(NVector, TimeVector, RateVector)  
  return(MyData) # Return the data frame  
}
```

Let's test it! You can play around with the model using the code below by changing the catch parameter. Try to see when the population crashes and when the population survives and refer back to the growth rate-abundance curve you plotted in our initial model.

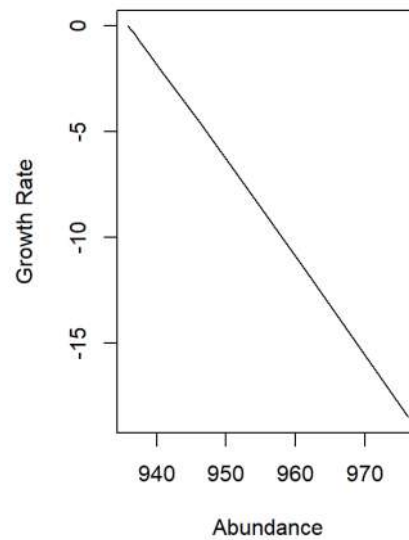
```
PlotCatchSimulation <- function(r,K,C,NStart,tEnd){  
  df <- CatchSimulator(r,K,C,NStart,tEnd)  
  par(mfrow=c(1,2)) # make a two panel plot of the next two graphs  
  PlotN(df)  
  PlotRate(df)  
}  
  
PlotCatchSimulation(r=0.5,K=1000,C=30,NStart=1000,tEnd=30)  
title("Figure 2.4: Simulated fixed catch quota dynamics",line=-1,outer=TRUE)
```

Figure 2.4: Simulated fixed catch quota dynamics

Change in Abundance Over Time



Rate of Change Curve



Quick Exercise: Try to find out the maximum sustainable yield (MSY) for the parameter values $r=0.5$, $K=1000$, by adjusting the catch parameter C until the population starts to crash.

Finding the MSY

Did you manage to find MSY by experimenting with the model? It is fine if you didn't, since there is a quantitative way to find MSY. As you can see from Figure 2.2, the MSY is basically the peak of the natural population growth curve. Conveniently, your `LogisticSimulator` function already outputs a data frame with all the data needed to plot the growth rate-abundance curve! To find the MSY, all you need to do is find the maximum rate of change from the data frame output of function `LogisticSimulator`.

Question 2.8

Write a piece of code that calculates the value of MSY for the parameter values $r=0.5$ and $K=1000$. To do this, run your logistic simulation (without harvesting) and find the value of the maximum growth rate (in the `RateVector`).

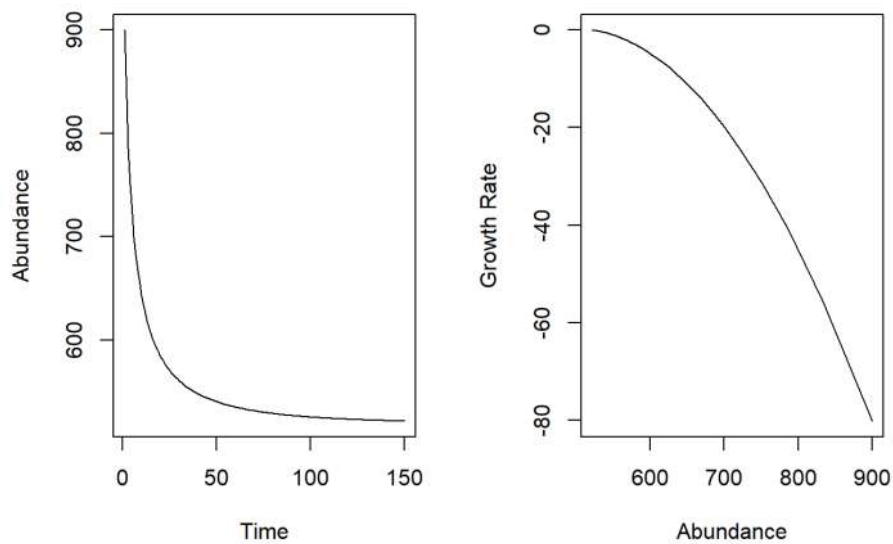
```
### QUESTION 2.8
SimulationResults2 <- LogisticSimulator(r=0.5, K=1000, NStart=1, tEnd=30)
MSY <- max(SimulationResults2$RateVector)
MSY
```

```
## [1] 124.8029
```

Let's simulate this scenario (for a longer period of time than before):

```
## DO NOT EDIT ##
PlotCatchSimulation(r=0.5, NStart=1000, K=1000, C=MSY, tEnd=150)
title("Figure 2.5: Harvesting at the maximum sustainable yield", line=-1, outer=TRUE)
```

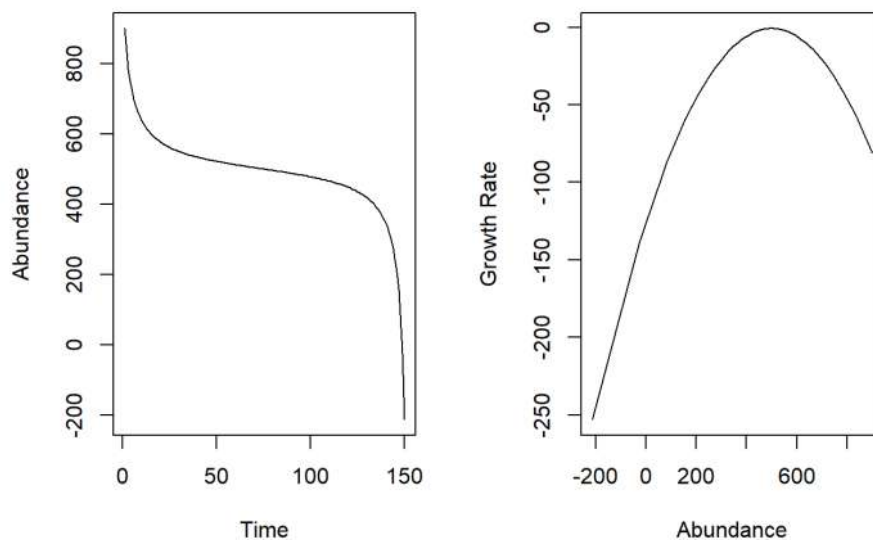
Figure 2.5: Harvesting at the maximum sustainable yield
 Change in Abundance Over Time Rate of Change Curve



As (hopefully!) seen from your simulation, catching at MSY won't lead to a population crash. Now, let's try increasing the catch to be 1 higher than the MSY and see how it affects the population trajectory.

```
## DO NOT EDIT ##
PlotCatchSimulation(r=0.5, NStart=1000, K=1000, C=MSY+1, tEnd=150)
title("Figure 2.6: Harvesting just above the maximum sustainable yield", line=-1, outer=TRUE)
```

Figure 2.6: Harvesting just above the maximum sustainable yield
 Change in Abundance Over Time Rate of Change Curve



Question 2.9

What are the possible consequences of trying to catch fish at the maximum sustainable yield? Write your answer below.

Catching fish at the maximum sustainable yield (MSY) is a concept in fisheries management that aims to harvest the maximum number of fish from a population without compromising its long-term sustainability. While the idea of maximizing yield may seem

appealing, there are several possible consequences and challenges associated with pursuing MSY:

1. **Overestimation of MSY:** Determining the exact MSY is challenging and often involves uncertainties in estimating population parameters, growth rates, and environmental factors. Overestimating MSY can lead to higher levels of exploitation than the population can sustain.
2. **Risk of Overexploitation:** Fishing at or near MSY levels for an extended period may result in overexploitation, where the fishing pressure exceeds the population's ability to replenish itself. This can lead to a decline in fish abundance, potentially compromising the long-term viability of the population.
3. **Selective Fishing Practices:** Achieving MSY might involve selective fishing practices that target specific age or size classes of fish. This can alter the population structure and composition, affecting the overall health and resilience of the population.
4. **Incomplete Knowledge:** Incomplete understanding of the biology and dynamics of fish populations, coupled with the complexities of marine ecosystems, can lead to unintended consequences when targeting MSY. Scientific uncertainty may limit the effectiveness of MSY-based management.
5. **Long-Term Sustainability Concerns:** MSY is a short-term concept, and focusing on maximizing yield without considering the long-term sustainability of fish stocks may lead to a decline in populations over time. Sustainable fisheries management requires a more comprehensive approach that considers ecological, social, and economic factors.

Challenge: Creating a fixed effort harvest model

The last thing we will try to model is another concept called fixed effort harvesting. This strategy controls the fishing effort of fishing boats (e.g certain days of the year), so that only a proportion of the total fish abundance is taken at every time step rather than a fixed quantity.

This is what the dynamics like:

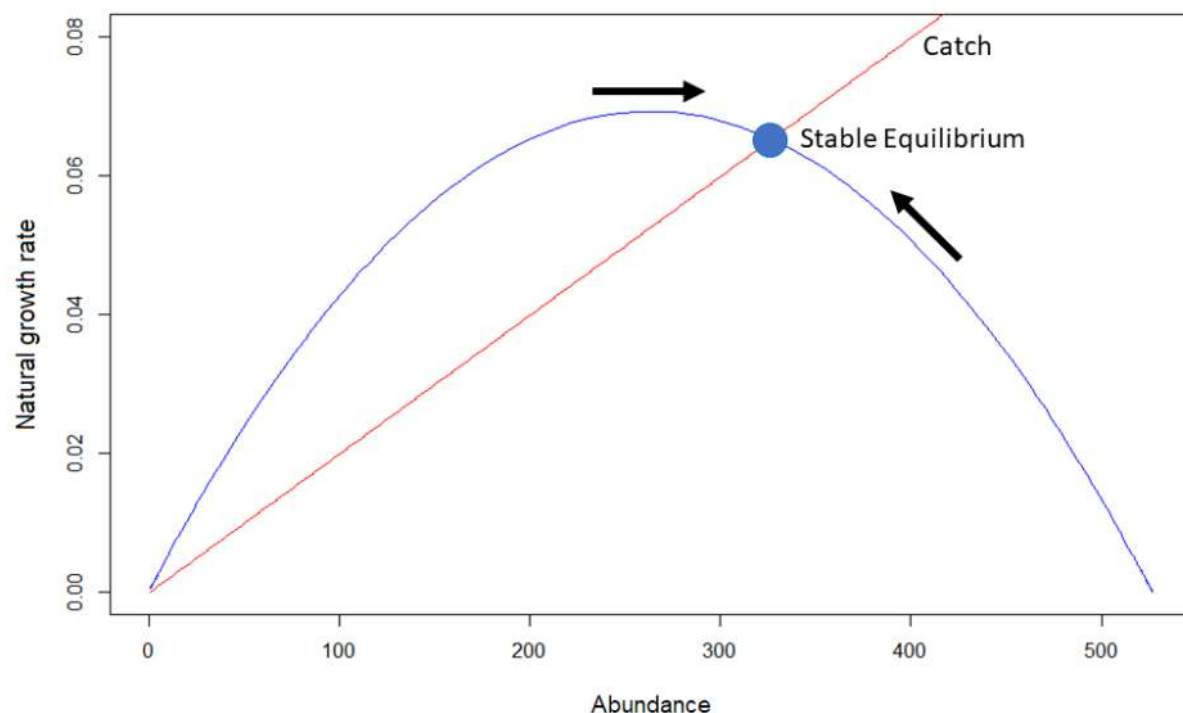


Figure 2.7: Behaviour of the logistic growth model with the addition of fixed-effort harvesting. Arrows represent the trajectory of the population.

Here is the formula for fixed effort harvesting.

$$\frac{dN}{dt} = rN \left(1 - \frac{N}{K} \right) - EN$$

where:

- r = intrinsic growth rate
- K = carrying capacity
- E = fixed effort
- N = population size

You could also write the same formula differently:

$$\frac{dN}{dt} = N \left(r \left(1 - \frac{N}{K} \right) - E \right)$$

Challenge Question 2.1

What do you see happening now and what are the implications of it in terms of sustainability? Answer the question by either writing text below, by writing more code in the chunk below, or both. Hint: Best to use and adapt the same code as you already got working above if you're going to attempt this.

```
### CHALLENGE QUESTION 2.1

# Modify the function with the new equation
RateOfChangeFixedEffort <- function(r, K, E, N) {
  dN_dt <- r * N * (1 - N / K) - E * N
  return(dN_dt)
}

NFixedEffort<-function(r,K,E,N,dt){
  output<-RateOfChangeFixedEffort(r,K,E,N)
  N<- N+dt*output
  return(N)
}

FixedEffortSimulator <- function(r, K, E, NStart, tEnd, dt = 0.01) {
  NVector <- c() # Initialize population abundance vector
  TimeVector <- c() # Initialize time vector
  RateVector <- c() # Initialize rate of change vector
  time <- 0 # Initialize time

  while (time <= tEnd) {

    for (i in 1:100){
      N <- NFixedEffort(r, K, E, NStart, dt)
      ratechange <- RateOfChangeFixedEffort(r, K, E, NStart)
      NStart <- N
      time <- time + dt
    }

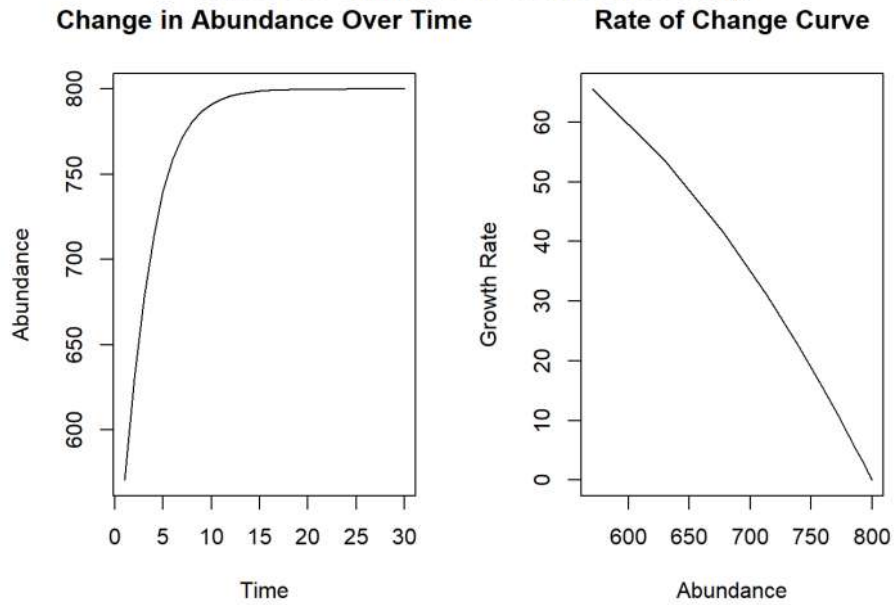
    NVector <- c(NVector, NStart)
    TimeVector <- c(TimeVector, time)
    RateVector <- c(RateVector, ratechange)
  }

  MyData <- data.frame(NVector, TimeVector, RateVector)
  return(MyData) # Return the data frame
}

PlotFixedEffortSimulator <- function(r, K, E, NStart, tEnd){
  df <- FixedEffortSimulator(r, K, E, NStart, tEnd, dt=0.01)
  par(mfrow=c(1,2)) # make a two panel plot of the next two graphs
  PlotN(df)
  PlotRate(df)
}

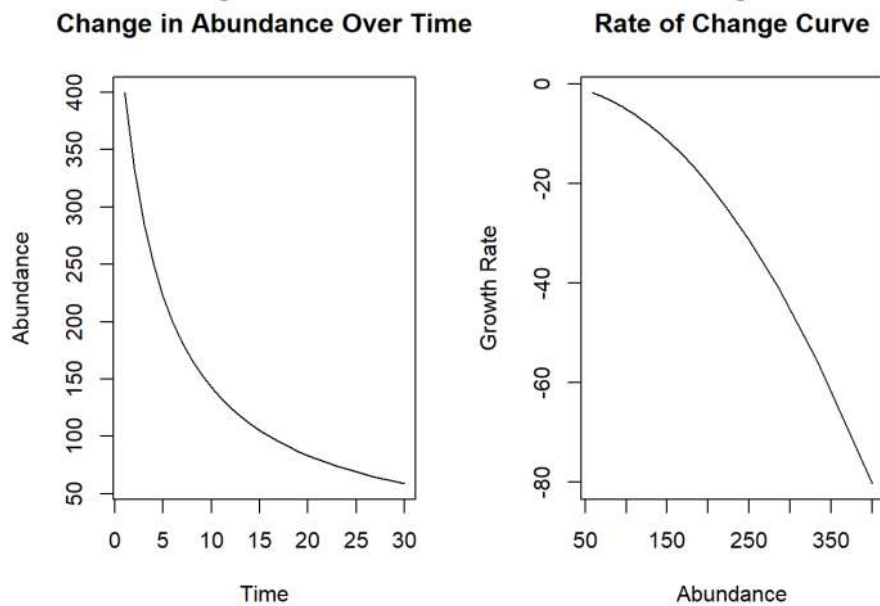
# Make two set of plot using different E values
PlotFixedEffortSimulator(r=0.5,K=1000,E=0.1,NStart=500,tEnd=30)
title("Figure 2.7: Simulated fixed effort harvesting",line=-1,outer=TRUE)
```

Figure 2.7: Simulated fixed effort harvesting



```
PlotFixedEffortSimulator(r=0.5,K=1000,E=0.5,NStart=500,tEnd=30)  
title("Figure 2.8: Simulated fixed effort harvesting",line=-1,outer=TRUE)
```

Figure 2.8: Simulated fixed effort harvesting



#In the modified simulation with fixed effort harvesting, lower E value increases the population abundance over time with higher increase of rate at first and then slows down.

#The higher increase in population abundance at first might indicate that the system is initially sensitive to harvesting, leading to a rapid decline in population size. This could be a result of overharvesting, where the harvesting process is more efficient in reducing the population initially.

#The slowdown in the rate of increase over time suggests that the population exhibits some level of resilience. As the population declines due to harvesting, it may adapt, recover, or develop mechanisms to cope with the ongoing harvesting pressure. This resilience is a positive aspect for the long-term sustainability of the population.

#In terms of sustainability, the goal is often to find a balance where harvesting meets human needs without compromising the long-term viability of the population. If the population abundance is increasing over time, it may suggest that the harvesting rate is not sustainable, and adjustments in harvesting practices or regulations may be necessary to avoid overexploitation.

Challenge Question 2.2

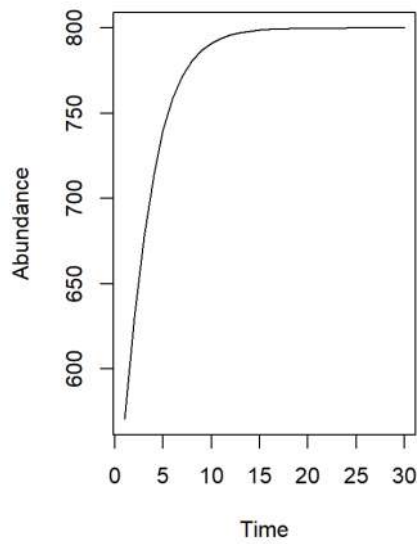
Add stochasticity to the intrinsic natural growth rate r of your population of fish using the same ideas as applied in the previous worksheet. What do you see happening now and what are the implications of it in terms of sustainability? Answer the question by either writing text below, by writing more code in the chunk below, or both. There is more than one way of approaching this question.

CHALLENGE QUESTION 2.2

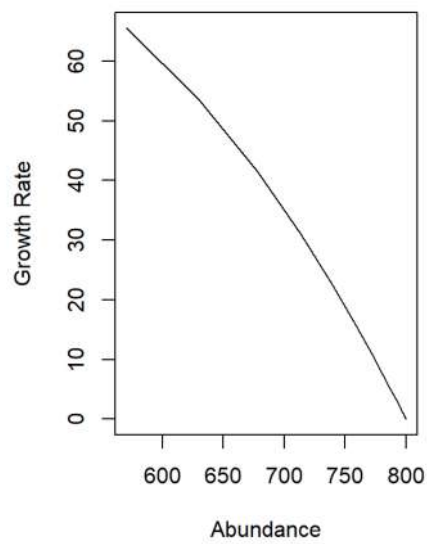
```
FixedEffortStochasticSimulator <- function(r, K, E, NStart, tEnd, dt = 0.01) {  
  NVector <- c()  
  TimeVector <- c()  
  RateVector <- c()  
  time <- 0  
  
  while (time <= tEnd) {  
  
    ## Generate random value of r within normal distribution using rnorm()  
    stochastic_r <- rnorm(1, mean = 0.5, sd = 0.2)  
  
    for (i in 1:100){  
      N <- NFixedEffort(stochastic_r, K, E, NStart, dt)  
      ratechange <- RateOfChangeFixedEffort(stochastic_r, K, E, NStart)  
      NStart <- N  
      time <- time + dt  
    }  
  
    NVector <- c(NVector, NStart)  
    TimeVector <- c(TimeVector, time)  
    RateVector <- c(RateVector, ratechange)  
  }  
  
  MyData <- data.frame(NVector, TimeVector, RateVector)  
  return(MyData)  
}  
  
PlotFixedEffortStochasticSimulator <- function(r, K, E, NStart, tEnd){  
  df <- FixedEffortStochasticSimulator(r, K, E, NStart, tEnd, dt=0.01)  
  par(mfrow=c(1,2))  
  PlotN(df)  
  PlotRate(df)  
}  
  
# Compare graphs without and with stochasticity using the function generated in Challenge 2.1  
  
PlotFixedEffortSimulator(r=0.5,K=1000,E=0.1,NStart=500,tEnd=30)  
title("Figure 2.9: Simulated fixed effort harvesting",line=-1,outer=TRUE)
```

Figure 2.9: Simulated fixed effort harvesting

Change in Abundance Over Time



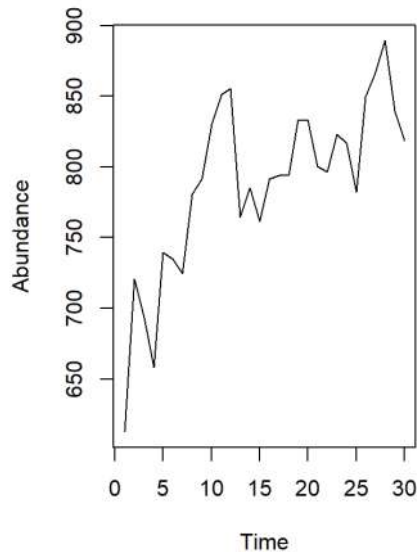
Rate of Change Curve



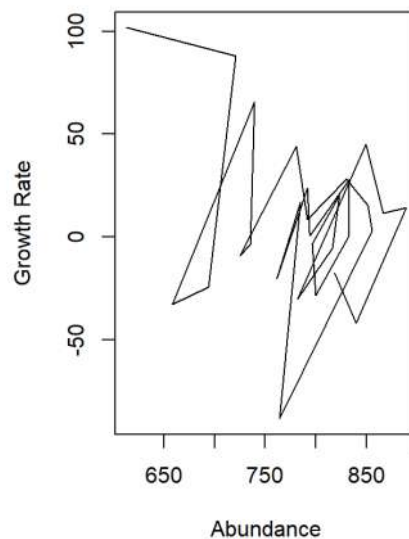
```
PlotFixedEffortStochasticSimulator(r=0.5,K=1000,E=0.1,NStart=500,tEnd=30)
title("Figure 2.10: Stochasticity Simulation",line=-1,outer=TRUE)
```

Figure 2.10: Stochasticity Simulation

Change in Abundance Over Time



Rate of Change Curve



#Stochasticity introduces random fluctuations, making it challenging to predict the exact trajectory of the population over time. In terms of sustainability, the implications are significant. Stochasticity can lead to unpredictable and potentially rapid changes in population abundance. This uncertainty poses challenges for fisheries management and conservation efforts. Sustainable harvesting practices often rely on stable and predictable population dynamics. The increased variability introduced by stochasticity may require adaptive management strategies to account for unforeseen fluctuations in population size and ensure the long-term sustainability of the fish population.

----- **End of document** -----

FINAL GRADE

93/100

GENERAL COMMENTS

Question 2.1 (out of 10)

10

excellent

Question 2.2 (out of 10)

10

excellent

Question 2.3 (out of 10)

10

excellent

Question 2.4 (out of 10)

10

excellent

Question 2.5 (out of 10)

10

excellent

Question 2.6 (out of 10)

10

excellent

Question 2.7 (out of 10)

10

excellent

Question 2.8 (out of 10)

10

excellent

Question 2.9 (out of 10)

7

in many ways an excellent answer but not tail

Total for main questions (out of 70)

67.67

Total for challenge questions (out of 15)

12.5

very good indeed, it would have been nice to

Quality of text answers (out of 5)

4

see my above comments

Quality of graphical outputs (out of 5)

5

exemplary

Quality of code (out of 5)

4

a bit weak on comments in places

final total

93

PAGE 1
PAGE 2
PAGE 3
PAGE 4
PAGE 5
PAGE 6
PAGE 7
PAGE 8
PAGE 9
PAGE 10
PAGE 11
PAGE 12
PAGE 13
PAGE 14
PAGE 15
PAGE 16
PAGE 17

GRADING FORM: LIFESCIGRADINGFORMMINORV1

YUKI WU

CRITERIA MET

How does this work meet the criteria for the grade it has been given?

AREAS TO IMPROVE

How could this work – and similar work in future – be improved, to gain a higher mark?

3-Competition-v2.html

by Yuki Wu

Submission date: 08-Feb-2024 09:24PM (UTC+0000)

Submission ID: 224043062

File name: 3-Competition-v2.html (1,006.94K)

Word count: 4269

Character count: 21005

3 - Resource use and competition

Essentials of Ecology coding coursework

Yuki Wu

February 2024

- Introduction
- Model specification
 - Modelling population change
 - **Question 3.1**
 - Modelling resource use
 - **Question 3.2**
 - Full model and plotting
 - **Question 3.3**
 - **Question 3.4**
- Two species: Competitive exclusion
 - **Question 3.5**
 - **Question 3.6**
 - **Question 3.7**
 - **Question 3.8**
 - **Challenge Question 3.1**

Introduction

Welcome to the third workbook! In this third exercise, we will produce another simple model to get more practice in writing mechanistic models in R. This time around, we will produce a model from scratch, based on a very important concept in ecology: resource use and competition. Please follow the instructions carefully!

Model specification

Modelling population change

Let's start by understanding the model we are trying to build. Our model will predict the consumption of a single resource by many competing species. For example, the resource could be sunlight, water, nutrients; basically anything.

To start, here is a baseline model of how resource availability influences the per-capita growth rate of a species. The overall growth rate of population size N is written $\frac{dN}{dt}$, a notation which should be familiar from the previous workbooks. We need the per-capita growth rate, that is the growth rate per individual organism, we can get this by dividing the rate by N to give:

$\frac{1}{N} \frac{dN}{dt}$ Let's consider how per-capita growth rate might be affected by differing amounts of a resource (Figure 3.1).

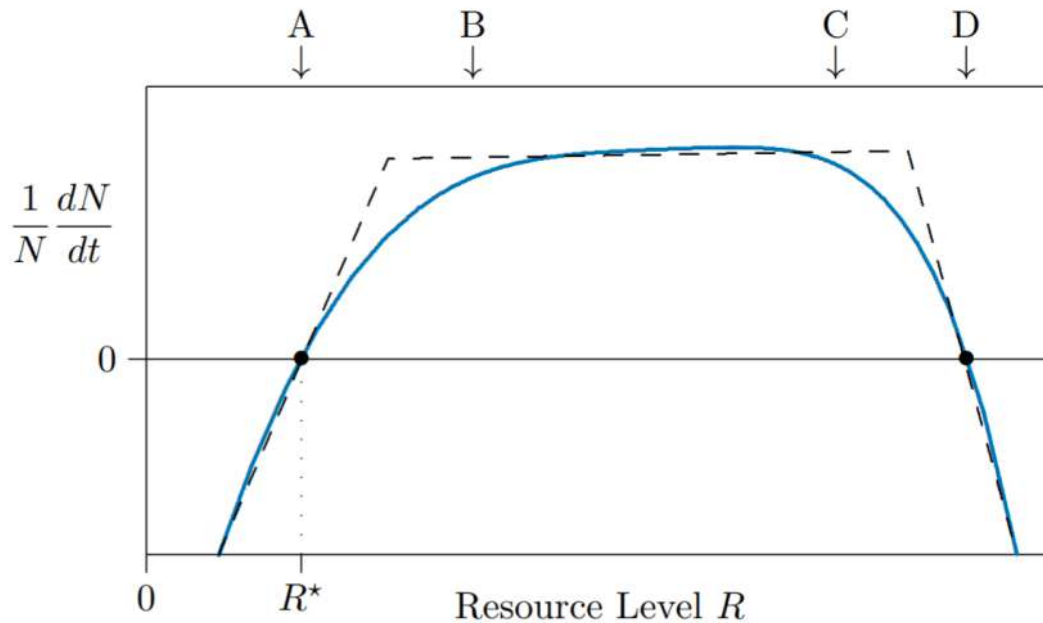


Figure 3.1: Model of how resource level affects per-capita growth rate.

To the left of label A, the resource level is too low, hence growth rate is negative (below zero) and the population will decline to extinction. To the right of label A, the growth rate increases as resource increases, until it reaches the maximum possible growth rate at B. After that, as the resource level becomes too abundant (after label C), it may lead to negative effects like over-heating (for sunlight) or over-saturation (for nutrients/water) leading to decreasing per-capita growth rates.

What does the figure tell us? When resources are abundant, growth rates are often high and resources are depleted from the environment. As a result, the resource level will drop until the growth rate reaches 0. The resource level at which growth rate would be 0 is called the R^* (pronounced "R star") of a species, and it characterises the species' competitiveness with respect to the resource. As you do more of this worksheet you will find out more.

For our model, we are only interested in the left hand side of the curve in Figure 3.1, since resources in the environment are usually used up really quickly and are rarely over-abundant. We can approximate this with a straight line (Figure 3.2).

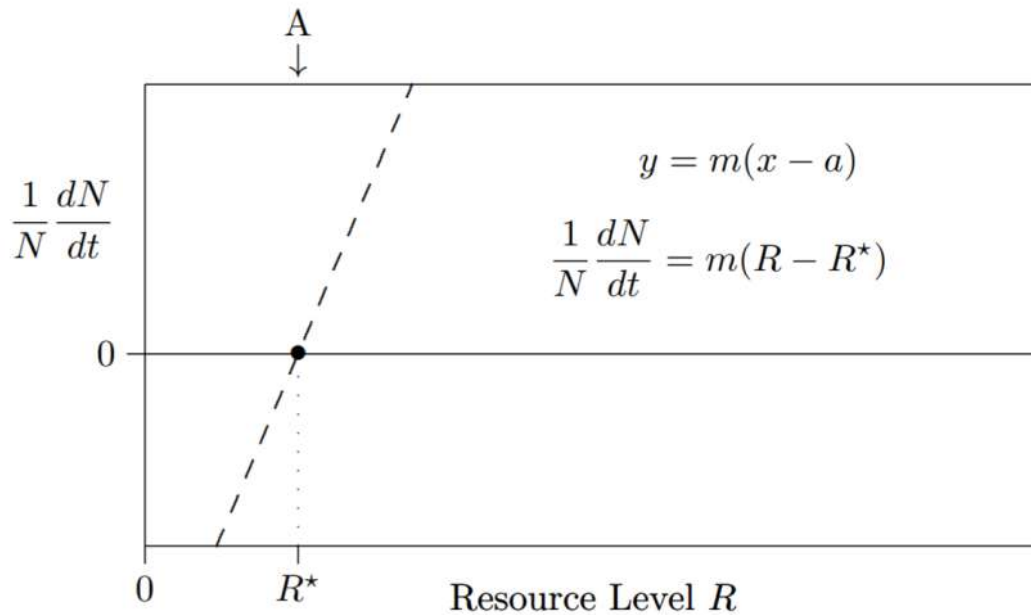


Figure 3.2: The linear portion of the resource use model.

The equation of the straight line provided in Figure 3.2, $y = m(x - a)$, is written in a form such that we can see the slope (m) and the x-intercept (a).

- The slope (how steep the line is) tells us how sensitive per-capita growth rate is to changes in resource availability. We will call m the growth constant.
- The x-intercept (where the line crosses the x-axis) tells us the threshold resource level at which per-capita growth rate switches between positive and negative. This is actually the R^* ("R star") mentioned earlier! Therefore, we will call this parameter R^* from now on.

Now we can write down the equation for the model. The per-capita growth of a species is defined by the excess resources available above its R^* ($R - R^*$), multiplied by a growth constant (m).

$$\frac{1}{N} \frac{dN}{dt} = m(R - R^*)$$

Doing some basic algebra, we find that:

$\frac{dN}{dt} = m(R - R^*)N$ Now we have the equation for the rate of change of the population, which would allow us to construct a model based on the equation!

Question 3.1

Create functions named `RateOfChangeN` and `ChangeN` to calculate what happens to the population size during a small amount of time dt has passed. `RateOfChangeN` will calculate the value of $\frac{dN}{dt}$ for given parameter and variable values, while `ChangeN` will calculate the small change in N during one time step. You may want to use the principles from the previous worksheets to help yourself along the way.

Function inputs:

- R^* (named `Rstar`)
- Growth constant (named `m`)
- Current population size (named `N`)
- Current resource level (named `R`)
- Size of time step (named `dt`) - only needed for the `ChangeN` function

Output:

- `RateOfChangeN` function: the value of $\frac{dN}{dt}$
- `ChangeN` function: the small change in N during one time step

Be very careful to remember that this function `ChangeN` outputs the **change** in N over one time step, not the new value of N after one time step has passed. This function is like the function `ChangeP` in worksheet 1, not like `LevinsStep` (worksheet 1) or `LogisticStep` (worksheet 2).

```
### QUESTION 3.1 ###
```

```
RateOfChangeN <- function(m, R, Rstar, N){
  dN_dt <- m*(R-Rstar)*N
  return(dN_dt)
}

ChangeN <- function(m, R, Rstar, N, dt){
  Change <- RateOfChangeN(m, R, Rstar, N)
  Output <- Change*dt
  return(Output)
}
```

Check if your functions work:

```
RateOfChangeN(Rstar=2, m=0.1, N=100, R=5)
```

```
## [1] 30
```

```
# Answer should be 30
```

```
ChangeN(Rstar=2, m=0.1, N=100, R=5, dt=0.01)
```

```
## [1] 0.3
```

```
# Answer should be 0.3
```

Modelling resource use

The second part of our model is based on resource use. In addition to calculating the change in population size, we also need to calculate and track the decrease in resource available in the system. This is a step on from anything you've done before because now you're going to have a change in resource use as well as in population. We will need to introduce a few more parameters. The first one is u , which is the amount of resource used by a single individual of a species.

In our model, we assume that an organism holds up a set amount of resources when it is alive, and immediately releases all those resources once it is dead. As such, the change in resources available depends on whether the population increases or decreases ($\frac{dN}{dt}$). When the population increases, more resources are used up; when the population decreases, resources are released back to the environment; and when population size is constant so are available resources. The rate of change of resource is thus connected to the rate of change of individuals and is given by:

$\frac{dR}{dt} = -u \frac{dN}{dt}$ This means we can say: $dR = -u \times dN$ So the change in resource (dR) is determined by the change in number of individuals (dN).

Question 3.2

Let's start putting this in our model. Firstly, we need to create a function (named `ChangeR`) that calculates the change in resource in the system given the change in number of individuals (dN) and resource consumption (u).

Function inputs: * Change in number of individuals (named `dN`) * Resource consumption (named `u`)

Output: * The change in resource level given by $dR = -u \times dN$

```
### QUESTION 3.2 ###
```

```
ChangeR <- function(dN, u){
  Output <- -u*dN
  return(Output)
}
```

Check your answer:

```
ChangeR(dN=10,u=0.1)
```

```
## [1] -1
```

```
# Answer should be -1
```

Full model and plotting

Question 3.3

You now need to write a function (named `ResourceSimulator`) that runs a while loop for a set amount of time (defined by `tEnd` as in previous worksheets), based on a starting population size `NStart` and a starting resource level `RStart`.

Within each loop, there are a few things you have to achieve:

- Increase the time by a small time step `dt` just as in your related answers given in previous worksheets.
- Calculate the updated population size `N` using the function `ChangeN`.
- Calculate the updated resource level `R` in the system using the function `ChangeR`.
 - Note: the input for the function `ChangeR` is the **CHANGE** in population size $\backslash(dN\backslash)$, which you already obtained using the function `ChangeN`.
- Record in separate vectors:
 - the time;
 - the population size (`N`) as it changes over time; and
 - the resource level (`R`) as it changes over time.
- Return these three vectors in a data frame using `data.frame` as you did in the last worksheet.
- You should use an inner loop, again as you did in the last worksheet. Make sure you edit this in at the end, so that it won't confuse you as you work through the other steps. Your inner for loop should repeat 10 times (not 100 times, as in this case there would be a noticeable drop in smoothness of the graph).

Tip: This time around, the input parameters required are not provided, try to figure it out as you work through the loop!

Tip 2: If you are very stuck, look a little down the worksheet and you'll see the code to test this function.

```
### QUESTION 3.3 ###

ResourceSimulator <- function(m, RStart, Rstar, NStart, dt, tEnd, u){
  TimeVector <- c()
  NVector <- c()
  RVector <- c()
  Time <- 0

  while(Time <= tEnd){
    Time <- Time + dt
    TimeVector <- c(TimeVector, Time)

    dN <- ChangeN(m, RStart, Rstar, NStart, dt)
    NStart <- dN + NStart
    NVector <- c(NVector, NStart)

    RStart <- ChangeR(dN, u) + RStart
    RVector <- c(RVector, RStart)
  }
  Output <- data.frame(NVector, RVector, TimeVector)
  return(Output)
}
```

Let's see the loop in action:

```
df <- ResourceSimulator(Rstar=5,m=0.5,u=0.01,NStart=50,RStart=10,tEnd=10,dt=0.01)
#print(df)
print(head(df, 10))
```

```
## NVector RVector TimeVector
## 1 51.25000 9.987500 0.01
## 2 52.52805 9.974720 0.02
## 3 53.83461 9.961654 0.03
## 4 55.17015 9.948298 0.04
## 5 56.53514 9.934649 0.05
## 6 57.93005 9.920700 0.06
## 7 59.35533 9.906447 0.07
## 8 60.81145 9.891886 0.08
## 9 62.29886 9.877011 0.09
## 10 63.81802 9.861820 0.10
```

Question 3.4

Lastly, we need to plot our output. For our model, we would like to plot the population size and resource level over time in the same graph. To do that, we will have to plot two different y-axes and two lines (best with different colours!).

- Create a function (named `PlotSimulation`) that performs a simulation by calling `ResourceSimulator` and then plots a graph of the two variables (population size $\backslash(N\backslash)$ and resource level $\backslash(R\backslash)$) through time.
- There must be two different y-axis with different scales, to the left and right of the plot.
- See below for an example plotting two y-axes, or search the internet!
- You need a legend as well so that your reader knows which line is which (you can see code for this in the plot in Worksheet 1, between Q1.4 and Q1.5).
- Give the graph an appropriate title and call it Figure 3.3.

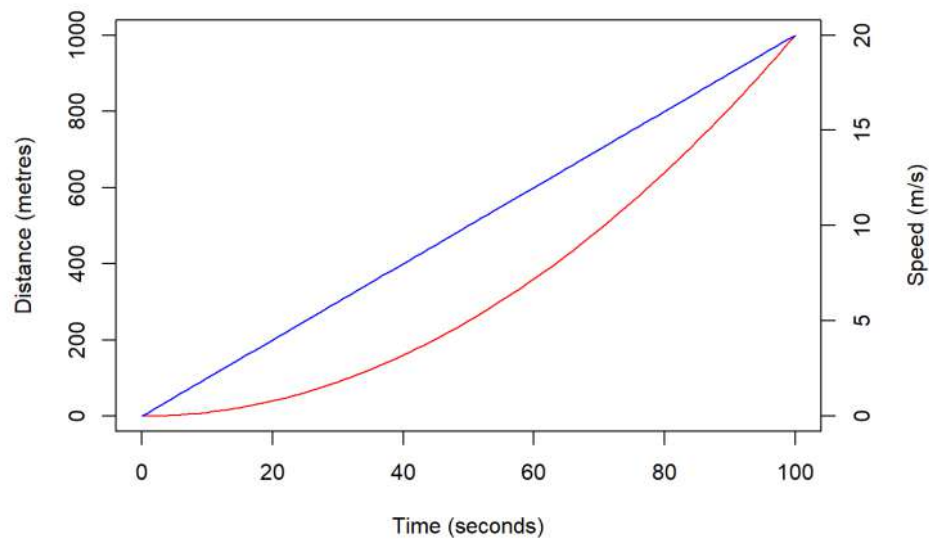
EXAMPLE CODE FOR PLOTTING GRAPHS WITH DIFFERENT y-AXES ON ONE PLOT:

```
# making some data to visualise
acceleration <- 0.2
time <- 0:100
speed <- time*acceleration
distance <- 0.5*time^2*acceleration

# make a plotting space
par(mar=c(5,4,4,4)+0.3)
# plot the distance graph
plot(time, distance, type="l", col="red",
      xlab="Time (seconds)", ylab="Distance (metres)")

# add a new plot
par(new=TRUE)
# plot the speed graph
plot(time, speed, type="l", col="blue",
      axes=FALSE, xlab="", ylab="")

# add an axis on the right hand side
axis(side=4, at=pretty(range(speed)))
# add the extra axis label
mtext("Speed (m/s)", side=4, line=3)
```



done!

Be careful when interpreting graphs with y-axes on different scales; when they cross it doesn't mean anything significant. In the graph above, it looks like distance and speed are equal at 100 seconds, when in fact at this time distance is 1km and speed is 20 m/s.

QUESTION 3.4

```
PlotSimulation <- function(m, RStart, Rstar, NStart, dt, tEnd, u){
  # create a dataframe
  data.frame <- ResourceSimulator(m, RStart, Rstar, NStart, dt, tEnd, u)

  # plot time on x-axis and population abundance on y-axis
  par(mar=c(5,4,4,4)+0.3)
  plot(data.frame$TimeVector, data.frame$NVector, type="l", col="red", xlab="Time (seconds)", ylab="Population Abundance")

  # plot another graph with resource abundance on y-axis
  par(new=TRUE)
  plot(data.frame$TimeVector, data.frame$RVector, type="l", col="blue", axes=FALSE, xlab="", ylab="")
  axis(side=4, at=pretty(range(data.frame$RVector)))
  mtext("Resource Abundance", side=4, line=3)

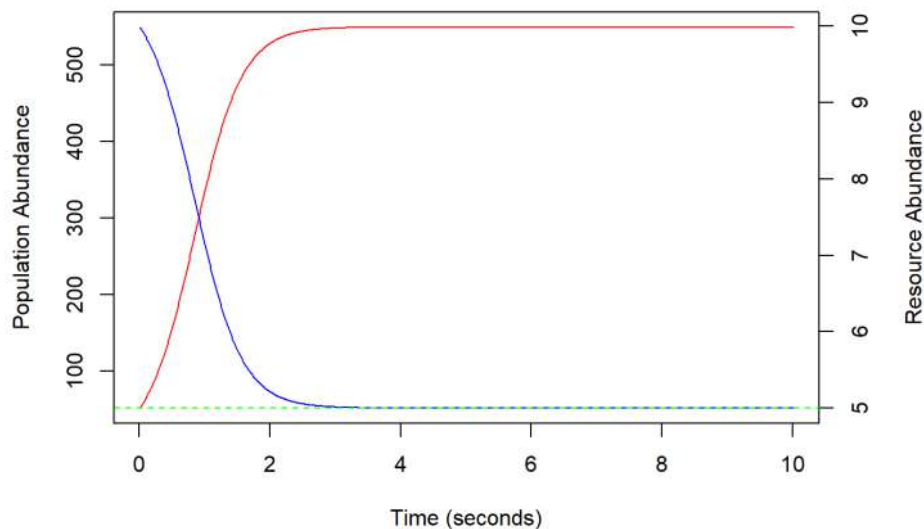
  # add a horizontal dashed line at the value of Rstar
  abline(h = Rstar, col = "green", lty = 2)

  # add title
  title("Figure 3.3", line=-1, outer=TRUE)
}
```

Now check your plot!

```
PlotSimulation(Rstar=5, m=0.5, u=0.01, NStart=50, RStart=10, tEnd=10, dt=0.01)
```


Figure 3.3



If you see an output graph, it works! You can play around with the parameters by changing the values and see how the output changes.

Here are a few things to confirm:

- The resource level will always stabilise at the species's R^* value, since there is no extra population growth once R^* is reached. (If you wish you could add a horizontal dashed line at the value of R^*).
- If the resource available is lower than the R^* , the population will decline and may go extinct. (You can check this by setting R_{start} less than R^*).

Congrats on completing the model! You've now built your own system for simulating systems of simultaneous ordinary differential equations - which is advanced. Now, we will move on by adding one more species into the model, so that the result of the competition between both species can be predicted.

Two species: Competitive exclusion

Question 3.5

Before we add one more species into the model, here are a few questions to think about:

Let's assume we have two species, with different R^* values.

- Will the species with a higher or lower R^* value have a greater competitive advantage?
- Which R^* value (higher or lower) will the resource level stabilise at and why?

If you're not sure of the answers to these questions to start with, this is fine! You will build the model and answer these questions yourself, so you can come back to this later. Write your answers as text below, and check back later to see if you want to change them.

Consider a scenario where Species A has a lower R^* value, indicating a more efficient resource utilization, while Species B has a higher R^* value, signifying a greater resource requirement. In competitive situations, Species A enjoys a significant advantage due to its lower resource threshold for growth. For instance, in conditions of low resource availability within the range of species A and B, species A thrives, while species B faces the risk of extinction.

In a competition between two species with different R^* values, the resource level will stabilize at the R^* value of the species with the lower R^* value. The reason behind this is that the species with the lower R^* value has a more efficient utilization of resources and can outcompete the other species for the available resources. As a result, the species with the lower R^* value will be able to maintain its population and keep the resource level stabilized at the point where its requirements are met. The species with the higher R^* value, on the other hand, will struggle to compete and may experience a decline in population, allowing the resource level to stabilize at the lower R^* value species' equilibrium.

Figure 3.4.2 serves as evidence supporting this observation. The green curve represents the species with the lower R^* , exhibiting a continuous increase in population abundance until it reaches its carrying capacity. In contrast, the red curve represents the species with the higher R^* , showing only a subtle increase in population abundance before declining to extinction. Consequently, the species with the lower R^* emerges victorious in the resource competition.

Now we are considering two species, let us denote their population sizes by N_1 (species 1) and N_2 (species 2). Most of the equations used to model two species will stay the same, except for the change in resource level. The change in resource will now need to incorporate species 2:

$$dR = -u_1 \times dN_1 - u_2 \times dN_2$$

Question 3.6

Copy down and then modify the `ChangeR` function from before, adding the extra component for the second species. Name this new function `ChangeRComp` (the `Comp` added on the end of the object name here is to indicate that this is the function for this new competition model). Hint: You will need two extra function inputs for the second species!

```
### QUESTION 3.6

ChangeRComp <- function(dN1, u1, dN2, u2){
  Output <- -u1 * dN1 - u2 * dN2
  return(Output)
}
```

Check if your function works correctly:

```
ChangeRComp(dN1=10,dN2=20,u1=0.1,u2=0.1)
```

```
## [1] -3
```

```
# Answer should be -3"
```

Question 3.7

Next, modify your `ResourceSimulator` function to include both species. Copy and paste your previous function into the box below, and start adding components for the second species. Call this new function `CompetitionSimulator`.

Your inputs will be:

- The parameter (R^*) for each species (`Rstar1` and `Rstar2`)
- The parameter (m) for each species (`m1` and `m2`)
- The parameter (u) for each species (`u1` and `u2`)
- The initial population size (N) for each species (`NStart1` and `NStart2`)
- The initial resource level (R) (`RStart`)
- The total amount of time to run the simulation for (`tEnd`)
- The size of the time step (`dt`)

The output of this new function should now be a dataframe with 4 columns; two columns recording the population size of each species (N_1 and N_2), one column for the resource available (R), and the one column for time.

Tip: Be mindful of the order of the calculation! Calculate dN first for each species (you can use the same `ChangeN` function for each, just call it with different parameters). Finally, update the resource, using the different dN s you calculated for each species and your new function `ChangeRComp`. It would be sensible to give your variables names showing which species number they relate to, to make sure you keep track of everything correctly.

QUESTION 3.7

```
CompetitionSimulator <- function(m1, m2, RStart, Rstar1, Rstar2, NStart1, NStart2, dt, tEnd, u1, u2){
  TimeVector <- c()
  N1Vector <- c()
  N2Vector <- c()
  RVector <- c()
  Time <- 0

  while(Time <= tEnd){
    Time <- Time + dt
    TimeVector <- c(TimeVector, Time)

    dN1 <- ChangeN(m1, RStart, Rstar1, NStart1, dt)
    NStart1 <- dN1 + NStart1
    N1Vector <- c(N1Vector, NStart1)

    dN2 <- ChangeN(m2, RStart, Rstar2, NStart2, dt)
    NStart2 <- dN2 + NStart2
    N2Vector <- c(N2Vector, NStart2)

    RStart <- ChangeRComp(dN1, u1, dN2, u2) + RStart
    RVector <- c(RVector, RStart)
  }
  Output <- data.frame(N1Vector, N2Vector, RVector, TimeVector)
  return(Output)
}
```

Test your function below.

```
df <- CompetitionSimulator(Rstar1=5,Rstar2=4,m1=0.3,m2=0.1,u1=0.001,u2=0.001,
  NStart1=1,NStart2=1,RStart=10,tEnd=2,dt=0.01)
```

```
#print(df)
```

```
print(head(df, 10))
```

```
##   N1Vector N2Vector RVector TimeVector
## 1  1.015000 1.006000 9.999979      0.01
## 2  1.030225 1.012036 9.999958      0.02
## 3  1.045678 1.018108 9.999936      0.03
## 4  1.061363 1.024217 9.999914      0.04
## 5  1.077283 1.030362 9.999892      0.05
## 6  1.093442 1.036544 9.999870      0.06
## 7  1.109843 1.042763 9.999847      0.07
## 8  1.126491 1.049020 9.999824      0.08
## 9  1.143387 1.055313 9.999801      0.09
## 10 1.160537 1.061645 9.999778      0.10
```

Question 3.8

Lastly, copy and modify your `PlotSimulation` function to create `PlotCompetition` which will plot both species. Again, copy and paste your previous function and modify it to add an extra line in the line graph for the second species. Give the graph an appropriate title and call it Figure 3.4.

You can plot one more line for species 2 using the function `lines`. Make sure you include this line with a unique colour and include this in the legend. You should add the argument `ylim=c(0,max(N1Vector,N2Vector))` into your first `plot` function, to ensure that the y-axis goes up to the maximum population size in the simulation. You would need to replace `N1Vector` and `N2Vector` with appropriate object names related to your own code.

QUESTION 3.8

```
PlotCompetition <- function(m1, m2, Rstart, Rstar1, Rstar2, Nstart1, Nstart2, dt, tEnd, u1, u2){
  # create a dataframe
  df <- CompetitionSimulator(m1, m2, Rstart, Rstar1, Rstar2, Nstart1, Nstart2, dt, tEnd, u1, u2)

  # plot time on x-axis and population abundance on y-axis
  par(mar=c(5,4,4,4)+0.3)
  plot(df$TimeVector, df$N1Vector, type="l", col="red", xlab="Time (seconds)", ylab="Population Abundance", ylim=c(0, max(df$N1Vector, df$N2Vector)))

  # overlay the second line
  lines(df$TimeVector, df$N2Vector, type="l", col="green")

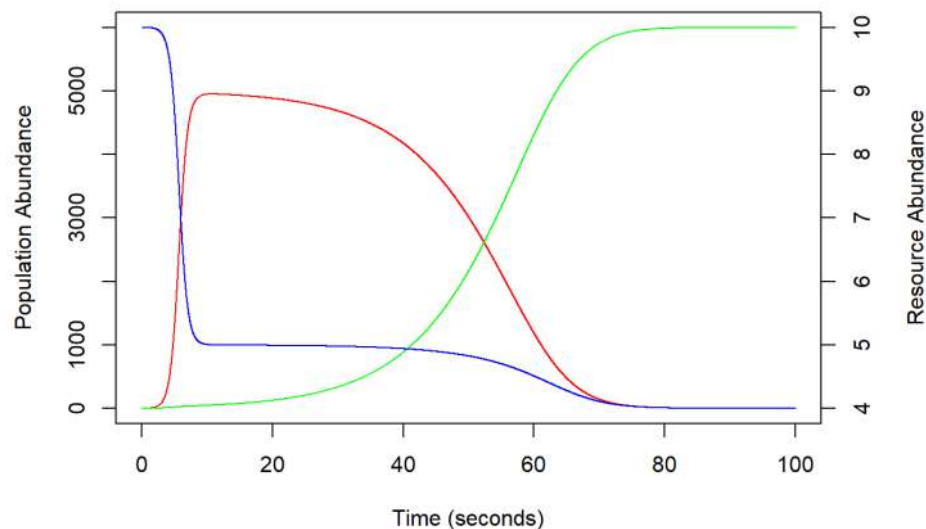
  # create a new plot for resource abundance
  par(new=TRUE)
  plot(df$TimeVector, df$RVector, type="l", col="blue", axes=FALSE, xlab="", ylab="")
  axis(side=4, at=pretty(range(df$RVector)))
  mtext("Resource Abundance", side=4, line=3)

  # add title
  title("Figure 3.4: Resource Competition", line=-1, outer=TRUE)
}
```

Now that you have all the components needed for the model, let's see if it works:

```
PlotCompetition(m1=0.3, m2=0.1, Rstart=10, Rstar1=5, Rstar2=4, Nstart1=1, Nstart2=1, dt=0.01, tEnd=100, u1=0.001, u2=0.001)
```

Figure 3.4: Resource Competition

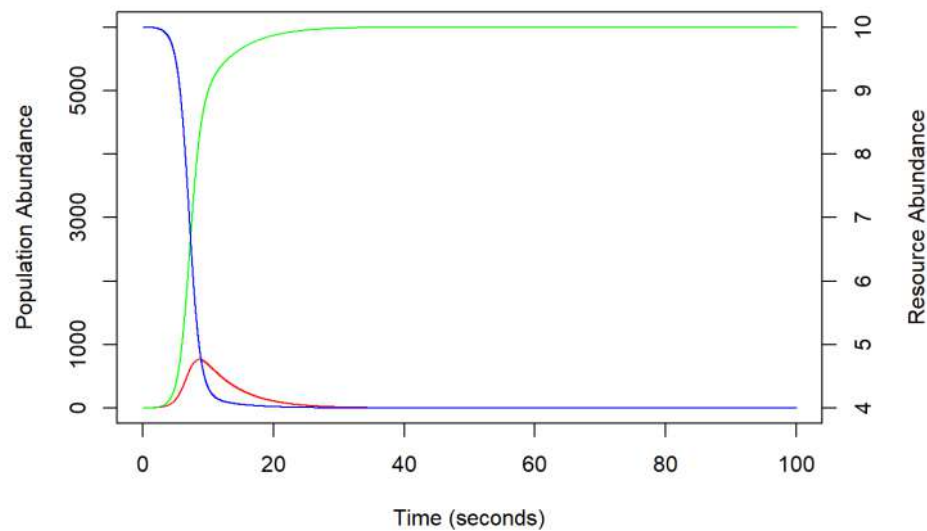


If you see an output graph that looks good, Congrats! In the code below, feel free to play around with the different parameters to see how each parameter changes the results. As you play with the model, try answer those questions introduced above (Question 3.5).

```
# Edit the parameters in this function to see how the dynamics change,
# and help you to answer Q3.5.
```

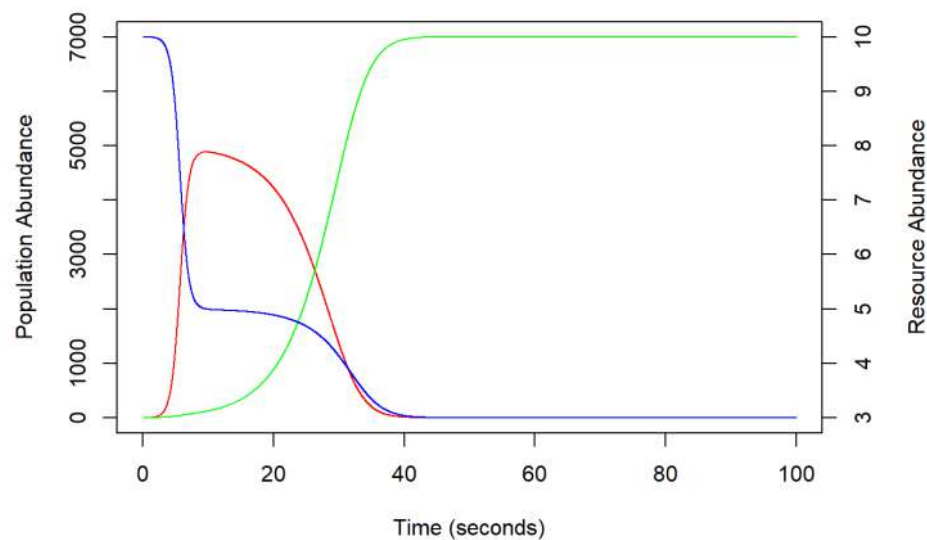
```
PlotCompetition(m1=0.2, m2=0.2, Rstart=10, Rstar1=5, Rstar2=4, Nstart1=1, Nstart2=1, dt=0.01, tEnd=100, u1=0.001, u2=0.001)
```

Figure 3.4: Resource Competition



```
PlotCompetition(m1=0.3, m2=0.1, RStart=10, Rstar1=5, Rstar2=3, NStart1=1, NStart2=1, dt=0.01, tEnd=100, u1=0.001, u2=0.001)
```

Figure 3.4: Resource Competition



Challenge Question 3.1

In this extra challenge, we want to add a time lag to our model, to allow species 1 to reach equilibrium first, before introducing species 2 into the environment.

This exercise is optional, like the other challenge questions, but there is extra credit available for doing it and it can be a good way for you to think of how mechanistic models in R can be modified to model different situations.

There are different ways to approach this problem, so try to think of a way to do this!

```
### CHALLENGE QUESTION 3.1 ###
```

```

CompetitionSimulatorWithLag <- function(m1, m2, RStart, Rstar1, Rstar2, NStart1, NStart2, dt, tEnd, u1, u2, lagTime){
  TimeVector <- c()
  N1Vector <- c()
  N2Vector <- c()
  RVector <- c()
  Time <- 0

  # Simulate for the lag time before introducing species 2
  while(Time <= lagTime){
    Time <- Time + dt
    TimeVector <- c(TimeVector, Time)

    dN1 <- ChangeN(m1, RStart, Rstar1, NStart1, dt)
    NStart1 <- dN1 + NStart1
    N1Vector <- c(N1Vector, NStart1)

    RStart <- ChangeRComp(dN1, u1, 0, 0) + RStart # Species 2 has no effect during the lag time
    RVector <- c(RVector, RStart)
  }

  # Continue simulation with competition between species 1 and 2
  while(Time <= tEnd){
    Time <- Time + dt
    TimeVector <- c(TimeVector, Time)

    dN1 <- ChangeN(m1, RStart, Rstar1, NStart1, dt)
    NStart1 <- dN1 + NStart1
    N1Vector <- c(N1Vector, NStart1)

    dN2 <- ChangeN(m2, RStart, Rstar2, NStart2, dt)
    NStart2 <- dN2 + NStart2
    N2Vector <- c(N2Vector, NStart2)

    RStart <- ChangeRComp(dN1, u1, dN2, u2) + RStart
    RVector <- c(RVector, RStart)
  }

  # Ensure that all vectors have the same length
  min_length <- min(length(N1Vector), length(N2Vector), length(RVector), length(TimeVector))
  Output <- data.frame(N1Vector[1:min_length], N2Vector[1:min_length], RVector[1:min_length], TimeVector[1:min_length])
  return(Output)
}

PlotCompetitionWithLag <- function(m1, m2, RStart, Rstar1, Rstar2, NStart1, NStart2, dt, tEnd, u1, u2, lagTime){
  # create a dataframe
  df <- CompetitionSimulatorWithLag(m1, m2, RStart, Rstar1, Rstar2, NStart1, NStart2, dt, tEnd, u1, u2, lagTime)

  # plot time on x-axis and population abundance on y-axis
  par(mar=c(5,4,4,4)+0.3)
  plot(df$TimeVector, df$N1Vector, type="l", col="red", xlab="Time (seconds)", ylab="Population Abundance", ylim=c(0, max(df$N1Vector, df$N2Vector)))

  # overlay the second line
  lines(df$TimeVector, df$N2Vector, type="l", col="green")

  # create a new plot for resource abundance
  par(new=TRUE)
  plot(df$TimeVector, df$RVector, type="l", col="blue", axes=FALSE, xlab="", ylab="")
  axis(side=4, at=pretty(range(df$RVector)))
  mtext("Resource Abundance", side=4, line=3)

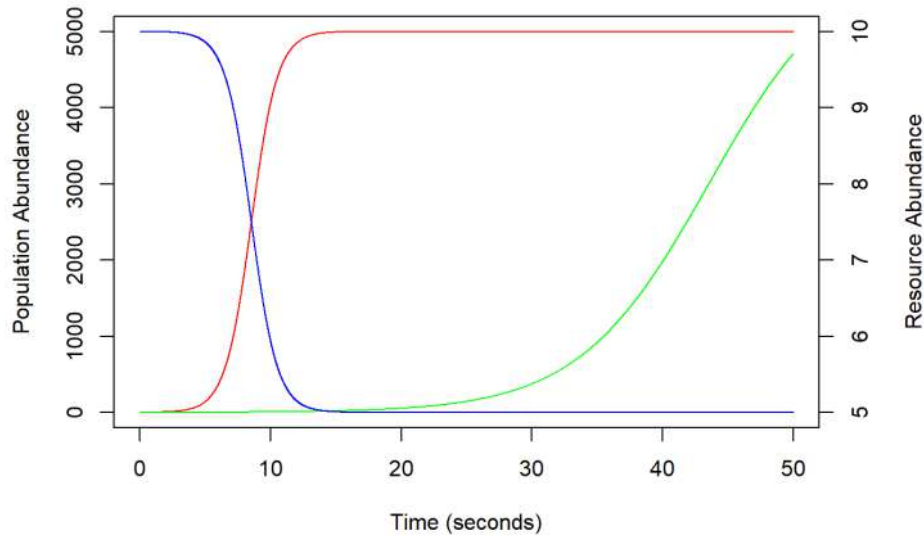
  # add title
  title("Figure 3.4: Resource Competition with Time Lag", line=-1, outer=TRUE)
}

PlotCompetitionWithLag(m1=0.2, m2=0.2, RStart=10, Rstar1=5, Rstar2=4, NStart1=1, NStart2=1, dt=0.01, tEnd=100, u1=0.001, u2=
0.001, lagTime=50)

```

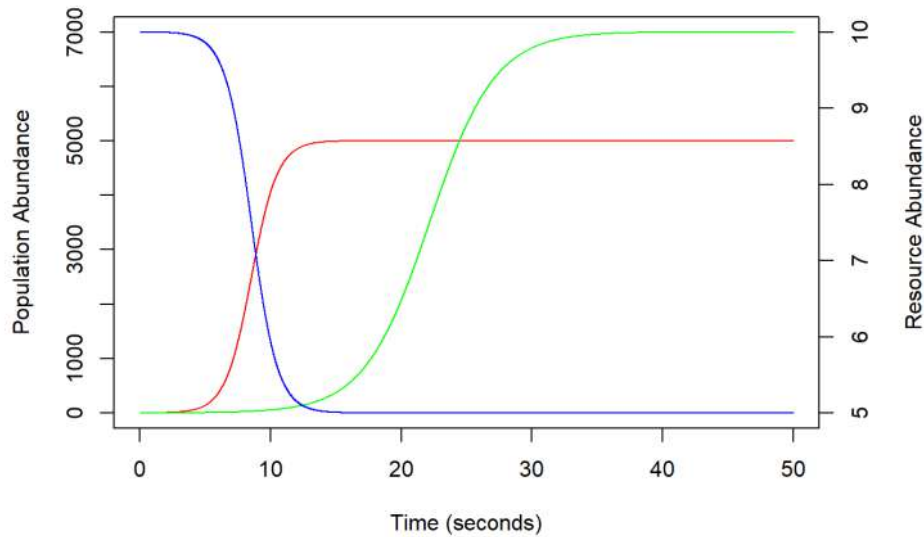
```
0.001, lagTime=50)
```

Figure 3.4: Resource Competition with Time Lag



```
PlotCompetitionWithLag(m1=0.2, m2=0.2, RStart=10, Rstar1=5, Rstar2=3, NStart1=1, NStart2=1, dt=0.01, tEnd=100, u1=0.001, u2=0.001, lagTime=50)
```

Figure 3.4: Resource Competition with Time Lag



Two graphs are presented, both indicating a lower R^* value for species 2. Initially, species 1 reaches equilibrium before the introduction of species 2. Subsequently, the abundance of species 2 surpasses that of species 1, establishing a higher population abundance due to its lower R^* value. Ultimately, both species coexist in the ecosystem. The lower R^* value the faster for species 2 to reach its equilibrium due to higher competitiveness.

Congrats on completing the third workbook! Throughout this notebook you learnt how to keep track of three different variables (two population sizes and resource level) throughout the model, and plot the results. This skill is really useful in the next and final

workbook, which is on predator and prey interactions.

----- **End of document** -----

FINAL GRADE

94/100

GENERAL COMMENTS

Question 3.1 (out of 10)

10
excellent

Question 3.2 (out of 10)

10
excellent

Question 3.3 (out of 10)

8
good but no inner loop

Question 3.4 (out of 10)

10
excellent

Question 3.5 (out of 10)

10
excellent

Question 3.6 (out of 10)

10
excellent

Question 3.7 (out of 10)

10
excellent

Question 3.8 (out of 10)

10
excellent

Total for main questions (out of 70)

68.25

Total for challenge questions (out of 15)

14
excellent, only possible improvement would b

Quality of text answers (out of 5)

5

excellent

Quality of graphical outputs (out of 5)

3.5

no legends

Quality of code (out of 5)

3.5

very sparse on comments, otherwise good

final total

94

PAGE 1

PAGE 2

PAGE 3

PAGE 4

PAGE 5

PAGE 6

PAGE 7

PAGE 8

PAGE 9

PAGE 10

PAGE 11

PAGE 12

PAGE 13

PAGE 14

PAGE 15

4-Predator-prey.html

by Yuki Wu

Submission date: 08-Feb-2024 09:27PM (UTC+0000)

Submission ID: 224043273

File name: 4-Predator-prey.html (860.43K)

Word count: 3219

Character count: 16005

4 - Lotka-Volterra predator-prey dynamics

Essentials of Ecology coding coursework

Yuki Wu

February 2024

- 4.1 Introduction
- Model specification
 - Prey
 - Predator
 - Summary
- Building the model
 - Question 4.1
 - Question 4.2
 - Question 4.3
 - Question 4.4
 - Question 4.5
 - Question 4.6
 - Question 4.7
- Challenge: Three species
 - Challenge Question 4.1

4.1 Introduction

Welcome to the final EoE coding worksheet! Here we will model a classic predator and prey model, called the Lotka-Volterra model. Since this our final exercise, we will provide less guidance. Look back to previous notebooks if you are struggling!

Model specification

There are two populations in the model, the prey (N) and the predator (P).

Here are the model assumptions:

- The predator's birth rate depends on the availability of prey, which they eat.
- The predator dies naturally, according to a natural mortality rate (q).
- In the absence of predation, the prey's population size follows exponential growth, with an intrinsic growth rate (r).
- Prey can only die through predation, which depends on the number of predators.

Prey

The **birth rate** of the prey will be rN , since this reflects exponential growth (birth rate is proportional to current population size). We will assume that the **mortality rate** is proportional to the number of predators, as well as the number of prey, so aNP . The parameter (a) reflects the search efficiency of predators.

From this, we can construct a differential equation describing the rate of change of prey population size, N :

$$\frac{dN}{dt} = rN - aNP$$

Predator

The **birth rate** of the predator depends on how much prey the predators are eating, and how this translates into reproduction. Recall that aNP is the amount of energy consumed by the predator population by eating prey. We then multiply this by an energy conversion index (c) to find out how this contributes to reproduction. As a result, the birth rate of the predators is $caNP$. The **mortality rate** is based on a constant per-capita natural mortality rate (q), so is qP .

From this, we obtain the equation:

$$\frac{dP}{dt} = caNP - qP$$

Summary

$$\frac{dN}{dt} = rN - aNP \quad \frac{dP}{dt} = caNP - qP$$

Variables:

- Prey population size (N)
- Predator population size (P)
- Time (t)

Parameters:

- Prey intrinsic growth rate (r)
- Predator mortality (q)
- Search efficiency (a) : The energy required to catch and handle prey
- Energy conversion index (c) : The proportion of energy converted to predator population growth from consuming prey

Building the model

We are at the final workbook, so by now you should be very familiar with building your own models. We will give you less guidance so you can try building this model yourself! Feel free to test your functions within the code chunks provided. You can check that some of functions give the right answer by doing the same calculation by hand.

Your Task:

Recreate the Lotka-Volterra predator and prey model in R following the equations above. Each question will ask for a different piece of code for the model, with the inputs already defined for you. The different possible inputs are listed below. Note that not all inputs will be required for every function.

List of possible input parameters:

- Current prey population size (named `N`)
- Current predator population size (named `P`)
- Initial prey population size (named `NStart`)
- Initial predator population size (named `PStart`)
- Prey growth rate (named `r`)
- Predator mortality (named `q`)
- Search efficiency (named `a`)
- Energy conversion index (named `c`)
- Simulation time (named `tEnd`)
- Small increase in time (named `dt`)

Question 4.1

Create a function `ChangeN` to calculate the change in prey population size (dN) over a small time step `dt`.

```
### QUESTION 4.1 ###

ChangeN <- function(r,N,a,P,dt){
  dN_dt <- r*N-a*N*P
  Output <- dN_dt*dt
  return(Output)
}
```

Question 4.2

Create the corresponding function `ChangeP` to calculate the change in population size (dP) for the predator.

```
### QUESTION 4.2 ###

ChangeP <- function(c,a,N,P,q,dt){
  dP_dt <- c*a*N*P-q*P
  Output <- dP_dt*dt
  return(Output)
}
```

Question 4.3

Create a function `PredationSimulator` to run a simulation of the whole system using the same approach as you've used before. Return the results (vectors containing population sizes for both predator and prey, and time) in a data frame as before. Use a default value of `dt=0.001` this time, but still have a length of 100 in your inner loop - so the vectors are updated every 100 small time steps.

Note: Make sure you use your functions `ChangeN` and `ChangeP` to compute dN and dP based on the previous time step values of N and P , not the current time step. The easiest way to keep track of this is to have an `N_old` and `P_old`, and `N_new` and `P_new`. In your loop, `N_old` and `P_old` are the population sizes at the previous time step, and you use these to compute dN and dP , which you then use to calculate `N_new` and `P_new`. Once you have computed both `N_new` and `P_new`, you can then overwrite the "old" values; `N_old <- N_new` and `P_old <- P_new`. By doing this, you can keep track of the population sizes at the last time step and the new time step, without accidentally overwriting them and getting your simulation out-of-sync.

QUESTION 4.3

```
PredationSimulator <- function(NStart,PStart,c,r,a,q,dt=0.001,tEnd){
  NVector <- c()
  PVector <- c()
  TimeVector <- c()
  Time <- 0
  N_old <- NStart
  P_old <- PStart

  while (Time <= tEnd){

    for (i in 1:100){
      N_new <- ChangeN(r,N_old,a,P_old,dt=0.001) + N_old
      P_new <- ChangeP(c,a,N_old,P_old,q,dt=0.001) + P_old
      N_old <- N_new
      P_old <- P_new
      Time <- Time + dt
    }

    NVector <- c(NVector, N_new)
    PVector <- c(PVector, P_new)
    TimeVector <- c(TimeVector, Time)
  }

  Output <- data.frame(NVector, PVector, TimeVector)
  return(Output)
}
```

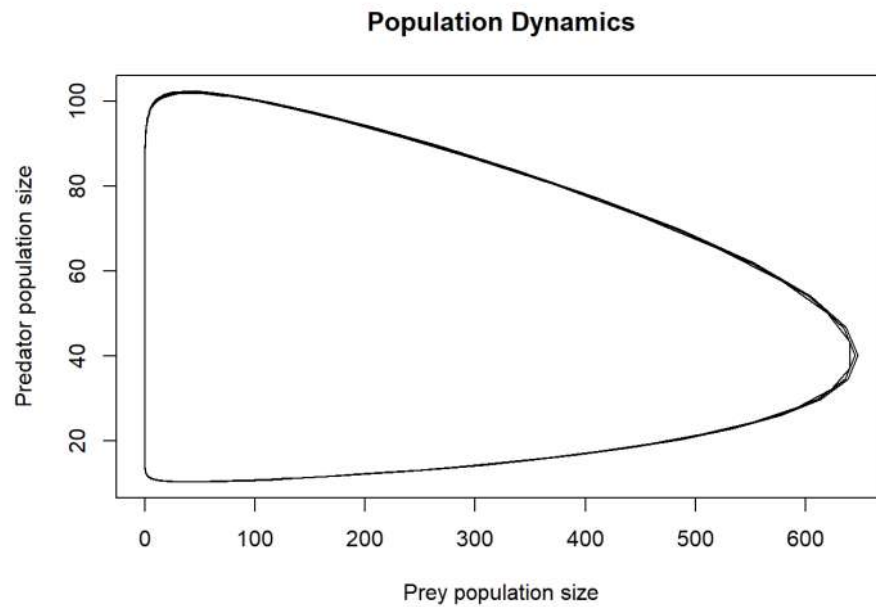
Question 4.4

Write a function called `PlotPvsN` to plot a graph with the prey population size on the x-axis and the predator population size on the y-axis. The function should take the data frame output of your simulation as its input. Ensure your plot has the necessary graphical features (e.g., appropriate axis labels), but it does not need to have a title.

QUESTION 4.4

```
PlotPvsN <- function(NStart,PStart,c,r,a,q,dt=0.001,tEnd){
  df <- PredationSimulator(NStart,PStart,c,r,a,q,dt=0.001,tEnd)
  plot(df$NVector, df$PVector, type = "l", xlab = "Prey population size", ylab = "Predator population size", main = "Population Dynamics")
}

PlotPvsN(NStart = 100, PStart = 100, c = 0.05, r = 2, a = 0.05, q = 0.1, dt=0.001, tEnd = 100)
```

Question 4.5

Write a function called `PlotPnt` to plot another graph, tracking changes in both prey and predator population sizes over time (so population sizes (P_t) and (N_t) go on the y-axis and time goes on the x-axis). Plot prey and predator lines in different colours and include a legend. The function should take the data frame output of your simulation as its input. Ensure your plot has the necessary graphical features (e.g., appropriate axis labels), but it does not need to have a title.

QUESTION 4.5

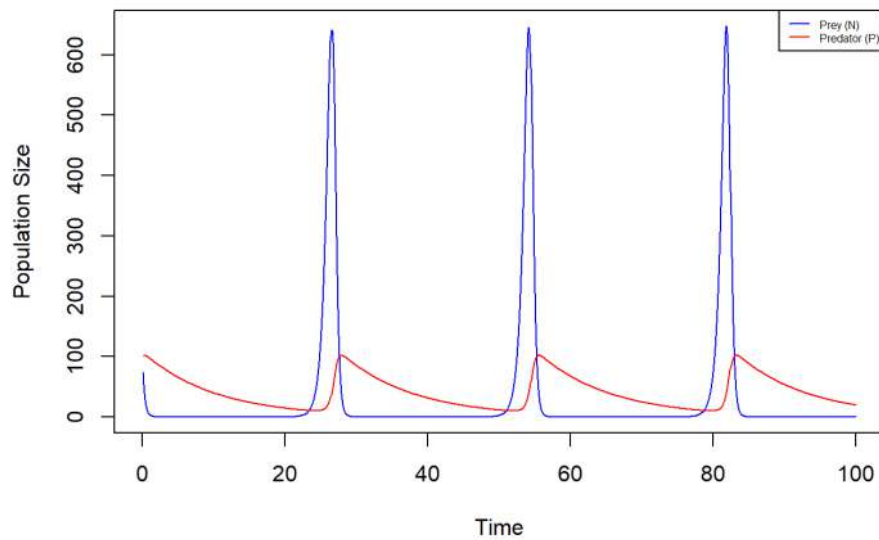
```
PlotPnt <- function(NStart,PStart,c,r,a,q,dt=0.001,tEnd) {
  simulation_df <- PredationSimulator(NStart,PStart,c,r,a,q,dt=0.001,tEnd)
  plot(simulation_df$TimeVector, simulation_df$NVector, type = "l", col = "blue",
       xlab = "Time", ylab = "Population Size", ylim = c(0, max(simulation_df$NVector, simulation_df$PVector)),
       main = "Population Dynamics over Time")

  lines(simulation_df$TimeVector, simulation_df$PVector, type = "l", col = "red")

  legend("topright", legend = c("Prey (N)", "Predator (P)"), col = c("blue", "red"), lty = 1, cex = 0.5, xpd = TRUE)
}

PlotPnt(NStart = 100, PStart = 100, c = 0.05, r = 2, a = 0.05, q = 0.1, dt=0.001, tEnd = 100)
```

Population Dynamics over Time



Question 4.6

You should have all the components you need to construct the model. Create a final function called `PlotPredation` to run the simulation and plot both of these graphs! The inputs should be all the necessary parameters to run a full simulation, and the code should plot the two graphs (`PlotPvsN` : (P) against (N) , and `PlotPNT` : (P) and (N) against (t)) together in a panel. To create a multi-figure panel you can either use `par(mfrow=c(1,2))` before plotting as you have seen done in previous worksheets, or you could look up the `layout` function for more customisable options.

QUESTION 4.6

```
PlotPredation <- function(NStart,PStart,c,r,a,q,dt=0.001,tEnd) {
  par(mfrow = c(1, 2)) # Set up a multi-figure panel

  # Plot P vs N
  PlotPvsN(NStart,PStart,c,r,a,q,dt=0.001,tEnd)

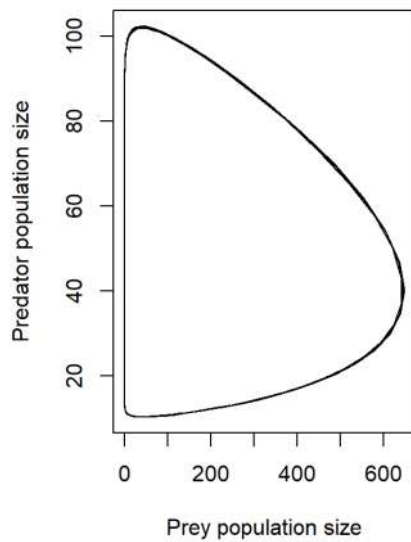
  # Plot P and N vs time
  PlotPNT(NStart,PStart,c,r,a,q,dt=0.001,tEnd)

  par(mfrow = c(1, 1)) # Reset to single-figure panel
}
```

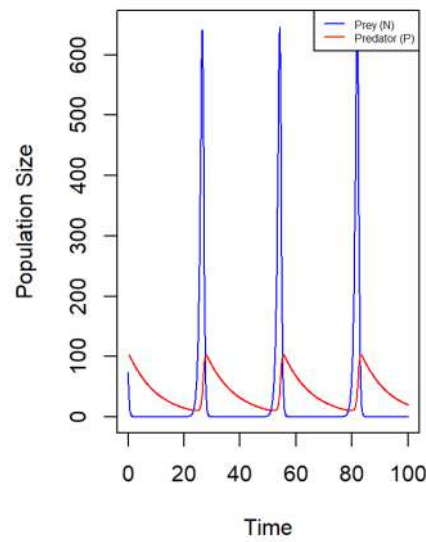
Let's see if your code works:

```
PlotPredation(NStart = 100, PStart = 100, c = 0.05, r = 2, a = 0.05, q = 0.1, dt=0.001, tEnd = 100)
```

Population Dynamics



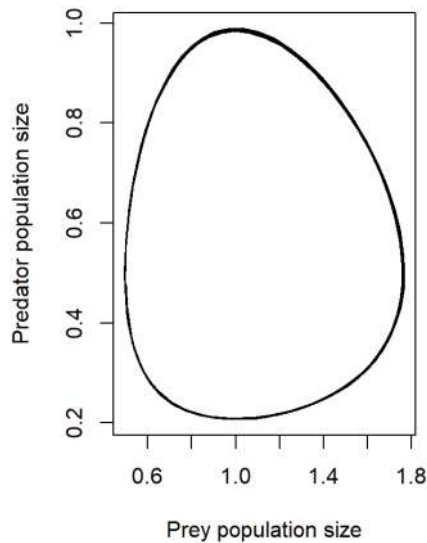
Population Dynamics over Time



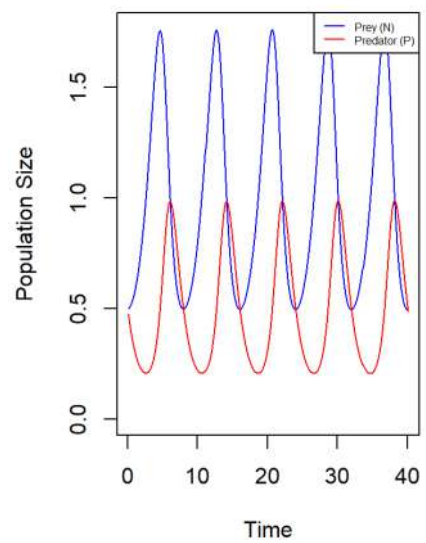
You can play around with the parameters to see how the periodic behaviour changes. The parameter values above and below should exhibit periodic behaviour.

```
PlotPredation(NStart = 0.5, PStart = 0.5, c = 3/4, r = 2/3, a = 4/3, q = 1, dt=0.001, tEnd = 40)
```

Population Dynamics



Population Dynamics over Time



Question 4.7

- What is the reason behind the periodic behaviour observed in the model?
 - The periodic behavior observed in the model is likely due to the interaction between the prey and predator populations. This interaction creates a dynamic system where the populations influence each other over time, leading to cycles of increase and decrease.
- What real-life factors are not considered in this model?
 1. Density-Dependent Effects: The model assumes that the predation rate is linearly proportional to the product of prey

and predator populations. In reality, the rate of predation may be influenced by the density of both populations, leading to more complex and nonlinear interactions.

2. Environmental Factors: The model does not consider the influence of external environmental factors, such as habitat quality, temperature, or other ecological conditions, which can significantly impact population dynamics.
 3. Age Structure and Life History: The model does not account for the age structure or life history characteristics of the populations. In real ecosystems, different age classes may experience different birth and mortality rates.
 4. Spatial Considerations: The model is a well-mixed system, and spatial considerations are not taken into account. Spatial heterogeneity, migration, and dispersal can have profound effects on population dynamics.
 5. Human Influence: Human activities, such as habitat destruction, pollution, and overexploitation, can significantly impact predator and prey populations. These anthropogenic factors are not considered in the model.
 6. Intraspecific Competition: The model assumes that all individuals within a population are equivalent. Intraspecific competition for resources and mates within each population is not explicitly included.
- How could some of these factors be included to increase the realism of the model?
 1. Density-Dependent Predation: Introduce a term in the predation equation that accounts for the density of both prey and predators. This could involve non-linear functions or more sophisticated functional responses (e.g., Holling's Type II or Type III).
 2. Environmental Factors: Add environmental parameters that affect birth and mortality rates. For example, introduce temperature-dependent growth rates or consider the impact of habitat quality on reproduction.
 3. Age Structure and Life History: Develop age-structured models with different equations for birth and mortality rates at different life stages. Consider incorporating life history traits, such as reproductive strategies and survival rates.
 4. Spatial Considerations: Implement spatially explicit models to account for the spatial distribution of populations, migration, and dispersal. This could involve partial differential equations to describe spatial dynamics.
 5. Human Influence: Integrate terms that represent human impacts on populations, such as harvesting rates, pollution effects, or habitat destruction. This requires additional parameters and equations to model anthropogenic influences.
 6. Intraspecific Competition: Modify the model to include intraspecific competition within each population. This could involve incorporating carrying capacity terms that account for resource limitations and competition among individuals.

[Write your answers here]

Challenge: Three species

Challenge Question 4.1

The final challenge (only recommended when you've done and carefully checked all other parts of all four worksheets) is to add one more species into the model as a top predator that preys on the predator from the previous model.

Here are the equations that define the model:

$$\text{Prey } (N): \frac{dN}{dt} = rN - a_1 P_1 N$$

$$\text{Predator 1 } (P_1): \frac{dP_1}{dt} = c_1 a_1 P_1 N - a_2 P_1 P_2 - q_1 P_1$$

$$\text{Predator 2 } (P_2): \frac{dP_2}{dt} = c_2 a_2 P_1 P_2 - q_2 P_2$$

where:

- a_1 and c_1 are the search efficiency and energy conversion index of predator 1 eating the prey
- a_2 and c_2 are the search efficiency and energy conversion index of predator 2 eating predator 1
- q_1 is the natural mortality rate for predator 1, q_2 is the natural mortality rate for predator 2
- r is still the intrinsic growth rate for the prey

Here are some interesting parameters to try if you get this model working: $(r=2, a_1=0.04, a_2=0.02, c_1=0.5, c_2=1, q_1=0.2, q_2=0.125)$. Try different starting values of N , P_1 and P_2 to observe the periodic behaviour.

Now you can build out the model yourself. For the output, plot graphs that show the population sizes of all three species against each other as well as their population size over time. You might like to go for several line graphs plots (predator 2 vs predator 1, predator 1 vs prey, all species vs time) or even try a 3D plot of prey vs predator1 vs predator2. It's up to you how you present your model, but ensure that you present, at the minimum, graph(s) showing predator 2 vs predator 1 and predator 1 vs prey, and a graph showing all three populations against time.

3D plot option: If you want to try an interactive 3D plot then I recommend you install an R package called `plotly`, which allows

you to plot line graphs in 3D. (If you do this, you can call the package using `require(plotly)` at the start of your code chunk.) This will let you track the change in population size of all 3 species and hopefully produce some cool interactive 3D plots! Using `plotly`, you can use the function `plotly::plot_ly(df,x=~N,y=~P1,z=~P2,type="scatter3d",mode="lines")`, making sure that you replace `df` with the name of your output data frame, and `N`, `P1`, and `P2` with the appropriate column names of your own data frame. Note that `plotly` plots will not render in RStudio - to view and interact with your plot you will need to open the HTML file in a compatible browser, such as Chrome.

Feel free to copy the answer code chunk to create multiple code chunks if this helps you set out your work.

```
### CHALLENGE QUESTION 4.1 ###

# Function to calculate the change in prey population over time
ChangeN <- function(r, N, a1, P1, dt = 0.001){
  dN_dt <- r * N - a1 * P1 * N
  Output <- dN_dt * dt
  return(Output)
}

# Function to calculate the change in Predator 1 population over time
ChangeP1 <- function(c1, a1, N, P1, a2, P2, q1, dt = 0.001){
  dP1_dt <- c1 * a1 * P1 * N - a2 * P1 * P2 - q1 * P1
  Output <- dP1_dt * dt
  return(Output)
}

# Function to calculate the change in Predator 2 population over time
ChangeP2 <- function(c2, a2, P1, P2, q2, dt = 0.001){
  dP2_dt <- c2 * a2 * P1 * P2 - q2 * P2
  Output <- dP2_dt * dt
  return(Output)
}

# Function to simulate the dynamics of three species over time
ThreeSpeciesSimulator <- function(N_start, P1_start, P2_start, r, a1, c1, a2, c2, q1, q2, dt = 0.001, tEnd){
  NVector <- c()
  P1Vector <- c()
  P2Vector <- c()
  TimeVector <- c()
  Time <- 0

  while (Time <= tEnd){

    for (i in 1:100){
      # Update population sizes based on differential equations
      N_new <- ChangeN(r, N_start, a1, P1_start, dt) + N_start
      P1_new <- ChangeP1(c1, a1, N_start, P1_start, a2, P2_start, q1, dt) + P1_start
      P2_new <- ChangeP2(c2, a2, P1_start, P2_start, q2, dt) + P2_start

      # Update initial values
      N_start <- N_new
      P1_start <- P1_new
      P2_start <- P2_new

      Time <- Time + dt
    }

    # Record population sizes at each time step
    NVector <- c(NVector, N_start)
    P1Vector <- c(P1Vector, P1_start)
    P2Vector <- c(P2Vector, P2_start)
    TimeVector <- c(TimeVector, Time)
  }

  # Create a dataframe with simulation results
  Output <- data.frame(NVector, P1Vector, P2Vector, TimeVector)
  return(Output)
}
```



```

# Function to plot Predator 1 vs Prey and Predator 2 vs Predator 1
Plot3SpeciesPN <- function(N_start, P1_start, P2_start, r, a1, c1, a2, c2, q1, q2, dt = 0.001, tEnd) {
  # Run simulation
  simulation_df <- ThreeSpeciesSimulator(N_start, P1_start, P2_start, r, a1, c1, a2, c2, q1, q2, dt, tEnd)

  # Plotting Predator 1 vs Prey
  par(mfrow=c(1,2)) # Set up a multi-figure panel
  plot(simulation_df$NVector, simulation_df$P1Vector, type = "l", col = "red",
       ylab = "Predator 1 Population Size", xlab = "Prey Population Size",
       main = "Predator 1 vs Prey Population")

  # Adding legend
  legend("topright", legend = "Predator 1 vs Prey", col = "red", lty = 1, cex=0.5)

  # Plotting Predator 2 vs Predator 1
  plot(simulation_df$P1Vector, simulation_df$P2Vector, type = "l", col = "green",
       ylab = "Predator 2 Population Size", xlab = "Predator 1 Population Size",
       main = "Predator 2 vs Predator 1")

  # Adding legend
  legend("topright", legend = "Predator 2 vs Predator 1", col = "green", lty = 1, cex=0.5)

  par(mfrow=c(1,1)) # Reset to single-figure panel
}

# Function to plot the dynamics of three species over time
Plot3SpeciesPNT <- function(N_start, P1_start, P2_start, r, a1, c1, a2, c2, q1, q2, dt = 0.001, tEnd) {
  # Run simulation
  simulation_df <- ThreeSpeciesSimulator(N_start, P1_start, P2_start, r, a1, c1, a2, c2, q1, q2, dt, tEnd)

  # Plotting prey and predators over time
  plot(simulation_df$TimeVector, simulation_df$NVector, type = "l", col = "blue",
       xlab = "Time", ylab = "Population Size", ylim = c(0, max(simulation_df$NVector)), main = "Population Dynamics over Time")

  lines(simulation_df$TimeVector, simulation_df$P1Vector, type = "l", col = "red")
  lines(simulation_df$TimeVector, simulation_df$P2Vector, type = "l", col = "green")

  # Add legend
  legend("topright", legend = c("Prey (N)", "Predator 1 (P1)", "Predator 2 (P2)"), col = c("blue", "red", "green"), lty = 1, cex=0.5)
}

# Combine two plots together into a function
PlotPredation2 <- function(N_start, P1_start, P2_start, r, a1, c1, a2, c2, q1, q2, dt = 0.001, tEnd) {
  par(mfrow = c(1, 2)) # Set up a multi-figure panel

  Plot3SpeciesPN(N_start, P1_start, P2_start, r, a1, c1, a2, c2, q1, q2, dt = 0.001, tEnd)

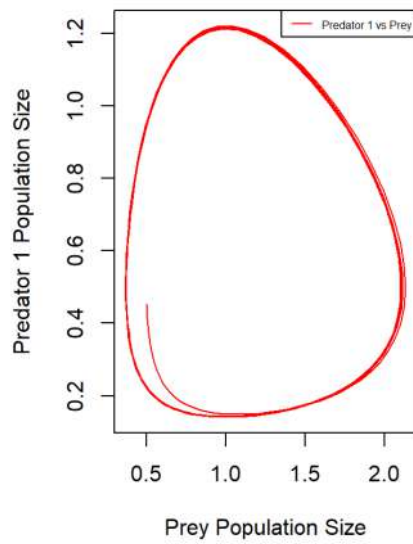
  Plot3SpeciesPNT(N_start, P1_start, P2_start, r, a1, c1, a2, c2, q1, q2, dt = 0.001, tEnd)

  par(mfrow = c(1, 1)) # Reset to single-figure panel
}

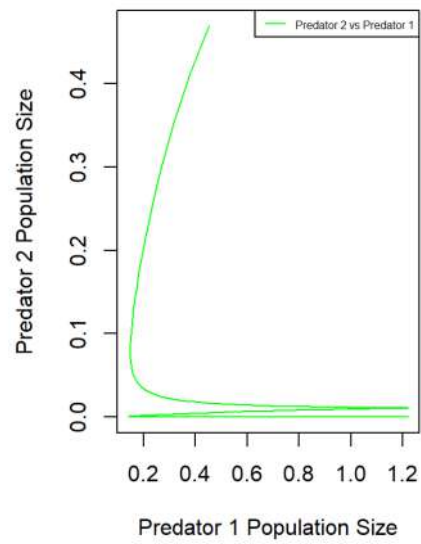
PlotPredation2(N_start=0.5, P1_start=0.5, P2_start=0.5, r=2/3, a1=4/3, c1=3/4, a2=3/3, c2=3/4, q1=1, q2=1, tEnd=40)

```

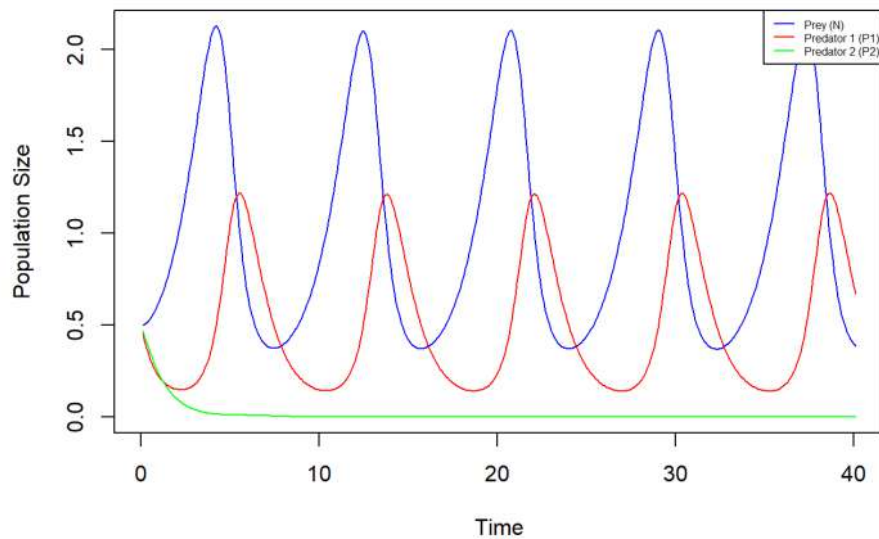

Predator 1 vs Prey Population



Predator 2 vs Predator 1

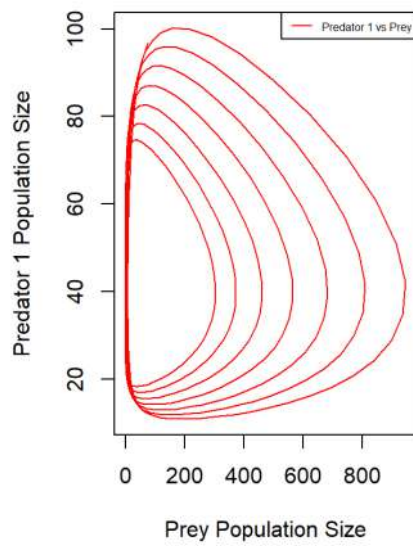


Population Dynamics over Time

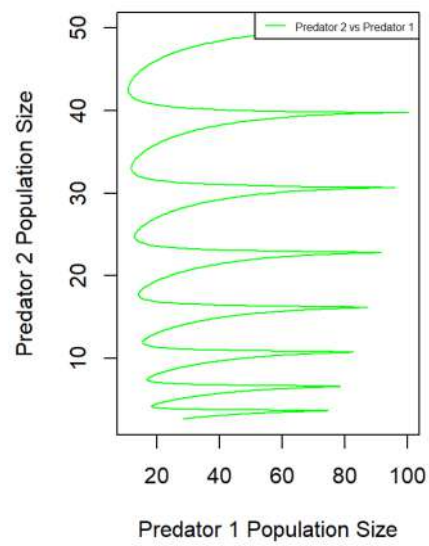


```
PlotPredation2(N_start=100, P1_start=100, P2_start=50, r=2, a1=0.05, c1=0.05, a2=0.01, c2=0.05, q1=0.05, q2=0.05, tEnd=100)
```

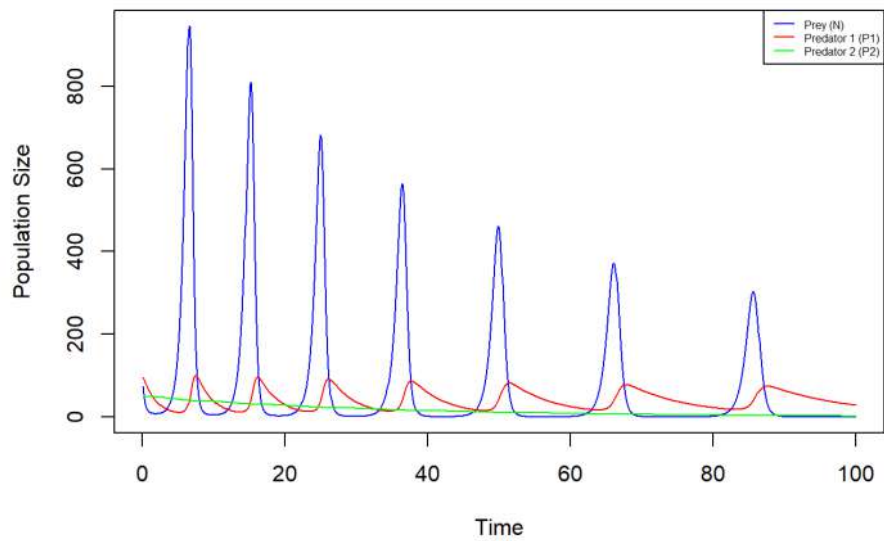
Predator 1 vs Prey Population



Predator 2 vs Predator 1



Population Dynamics over Time



----- *End of document* -----

FINAL GRADE

93/100

GENERAL COMMENTS

Question 4.1 (out of 10)

10
excellent

Question 4.2 (out of 10)

10
excellent

Question 4.3 (out of 10)

10
excellent

Question 4.4 (out of 10)

10
excellent

Question 4.5 (out of 10)

10
excellent

Question 4.6 (out of 10)

10
excellent

Question 4.7 (out of 10)

9
excellent, though I would have liked a bit mor

Total for main questions (out of 70)

69

Total for challenge questions (out of 15)

11
very good, would have been nice to add the 3

Quality of text answers (out of 5)

5
excellent

Quality of graphical outputs (out of 5)

4

good though the legend is badly placed and c

Quality of code (out of 5)

4

tidy but sparsely commented

final total

93

PAGE 1
PAGE 2
PAGE 3
PAGE 4
PAGE 5
PAGE 6
PAGE 7
PAGE 8
PAGE 9
PAGE 10
PAGE 11